

BiTrajDiff: Bidirectional Trajectory Generation with Diffusion Models for Offline Reinforcement Learning

Yunpeng Qing^{*1} Yixiao Chi^{*1} Shuo Chen^{*1} Shunyu Liu² Kexuan Zhou¹ Sixu Lin³ Litao Liu⁴
Changqing Zou^{†51}

Abstract

Offline Reinforcement Learning (RL) relies on static datasets and often enforces conservative constraints to mitigate out-of-distribution errors, but this inevitably gives rise to learning dataset biases and limited behavioral generalization. Recent Data Augmentation (DA) methods leverage generative models to enrich offline data, yet they mainly operate within a single rollout paradigm and tend to preserve the original trajectory-level connectivity of the dataset. As a result, such methods often introduce local variations and fail to recover connections between distinct behavior patterns. In this paper, we propose Bidirectional Trajectory Diffusion (BiTrajDiff), a novel DA framework that explicitly addresses this limitation. BiTrajDiff decomposes trajectory synthesis into two independent diffusion processes that generate forward-future and backward-history segments conditioned on shared intermediate anchor states. By stitching the generated segments at these anchors, BiTrajDiff can synthesize trajectories that bridge disconnected behavior patterns and recover global trajectory-level connectivity absent from the original data. Extensive experiments demonstrate that BiTrajDiff consistently outperforms advanced DA methods across a range of offline RL backbones. Our code is available at <https://github.com/Plankson/BiTrajDiff>.

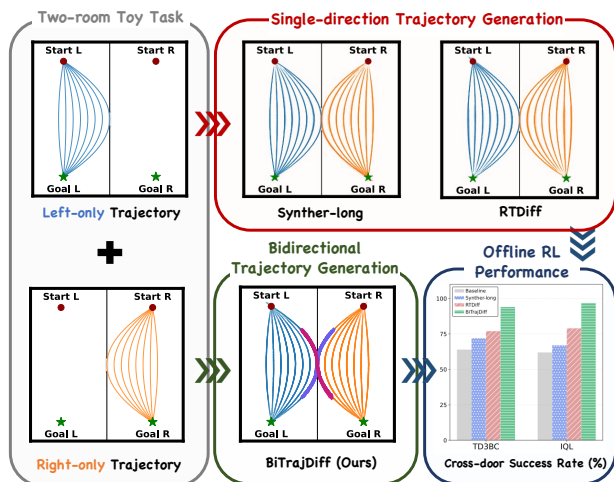


Figure 1. Two-room toy task with the offline dataset. The dataset contains **left-only** and **right-only** trajectories and lacks cross-door pattern; single-direction generation fits the in-room distribution but rarely bridges the doorway. BiTrajDiff generates **cross-door** trajectories and improves offline RL cross-door success.

1. Introduction

Offline Reinforcement Learning (Offline RL) (Fujimoto et al., 2019) seeks to derive effective decision policies solely from pre-collected datasets, obviating the necessity for on-line exploration due to the inherent risks and costs associated with real-time interaction (Levine et al., 2020). This paradigm has shown strong potential in real-world domains such as robotic control (Qing et al., 2024; 2022; Liu et al., 2023; Kong et al., 2024; Liu et al., 2024a) and power grid management (Chen et al., 2024; Xu et al., 2024). A primary challenge in offline RL is the out-of-distribution (OOD) problem: during training, the learned policy induces a state-action distribution that can deviate from the support of the logged data, causing extrapolation error and value overestimation (Levine et al., 2020). To mitigate this, many offline RL methods enforce conservatism by imposing explicit constraint terms during policy optimization (Qing et al., 2024; Chen et al., 2022) and penalizing the value estimation of OOD actions (Lyu et al., 2022; Xu et al., 2023). However, such conservatism also tightly induces learning to the

^{*}Equal contribution ; [†]Corresponding author. ¹Zhejiang University ²Nanyang Technological University ³The Chinese University of Hong Kong (Shenzhen) ⁴Rutgers University-New Brunswick ⁵Zhejiang Lab. Correspondence to: Changqing Zou <changqing.zou@zju.edu.cn>.

structural biases of the offline dataset, limiting behavioral diversity and generalization beyond the logged trajectories.

To alleviate the structural bias imposed by conservative offline learning, a natural solution, Data Augmentation (DA), enriches the offline dataset with synthesized transitions while remaining orthogonal to the underlying offline RL optimization procedure. Recent works (Lu et al., 2023; Li et al., 2024; Li & Zhang, 2024; Su et al., 2023) leverage generative models, particularly diffusion models, to synthesize high-fidelity trajectories for offline RL, yielding promising empirical improvements over existing offline RL algorithms. However, existing DA methods typically generate trajectories within a single rollout paradigm (forward-future (Lee et al., 2024) or backward-history (Yang & Wang, 2025)) conditioned on in-dataset states. Therefore, the synthesized data remain confined to the original trajectory distribution, rather than creating novel trajectories connecting previously disconnected states in the dataset. Consequently, the augmented data inevitably ignores the connectivity between various behavior patterns and fails to synthesize trajectories that explicitly bridge them. To exemplify the above issue, we conduct a didactic experiment using representative single-direction data augmentation methods, including Synther-long, a trajectory-level extension of Synther (Lu et al., 2023), and RTDiff (Yang & Wang, 2025), in a toy task shown in Figure 1. The offline dataset contains two distinct trajectory sets confined to each side of the wall and includes no cross-door trajectories, despite covering many states near the narrow doorway. The generation results show that the methods both successfully fit the distribution of the original data, capturing within-room variations, while rarely synthesizing trajectories that traverse the door to connect the two patterns. This indicates that single-process rollouts tend to preserve the original trajectory-level connectivity, resulting in limited performance improvement.

In this paper, we propose Bidirectional Trajectory Diffusion (BiTrajDiff), a novel DA framework that explicitly addresses the trajectory-level connectivity limitation of existing DA methods. Unlike prior approaches that rely on a single rollout paradigm, BiTrajDiff decomposes trajectory synthesis into two independent yet complementary diffusion generation processes: one modeling forward-future trajectories and the other modeling backward-history trajectories. Crucially, these two processes are conditioned on the same intermediate anchor states sampled from the dataset, enabling the independent exploration of plausible pasts and futures around these anchors. Therefore, BiTrajDiff can synthesize trajectories that bridge distinct behavior patterns and recover connections absent from the original dataset. As evidenced in Figure 1, despite the absence of any cross-door trajectories in the dataset, BiTrajDiff synthesizes such trajectories and achieves higher performance than the single-direction DA baselines. Technically, we implement

BiTrajDiff with a forward trajectory diffusion model conditioned on terminal states and a backward trajectory diffusion model conditioned on initial states, followed by a stitching procedure that concatenates the generated segments into complete trajectories. Meanwhile, we incorporate a lightweight quality-control step to discard low-confidence synthesized trajectories. As a result, BiTrajDiff expands the offline dataset not only through local variations but also by recovering global trajectory-level connections, leading to substantially improved performance across a wide range of offline RL algorithms.

Our contributions are summarized as follows:

- We identify a fundamental limitation of existing DA methods for enhancing offline RL algorithms: single rollout processes tend to preserve the original trajectory-level connectivity of the dataset, failing to explore the ways to connect distinct behavior patterns.
- We propose Bidirectional Trajectory Diffusion (BiTrajDiff), which decomposes trajectory synthesis into independent forward-future and backward-history diffusion processes, and then explicitly stitches the generated segments at shared intermediate anchor states, enabling the synthesis of trajectories that connect previously disconnected behavior patterns.
- We demonstrate the effectiveness of BiTrajDiff through extensive experiments on D4RL across multiple offline RL backbones, where it consistently outperforms various types of DA baselines, validating its performance gains and robustness.

2. Related Works

Offline RL encompasses four primary categories: policy constraint (Wang et al., 2022; Xu et al., 2023), value regularization (Kostrikov et al., 2021; Kumar et al., 2020), model-based (Yu et al., 2020; Chemingui et al., 2024), and return-conditioned supervised learning (Chen et al., 2021; Paster et al., 2022). Policy constraint methods restrict policies to the offline dataset, employing techniques such as explicit behavior cloning regularization (Qing et al., 2024; Chen et al., 2022) or implicit optimization towards in-sample optimal policies (Nair et al., 2020; Yue et al., 2022). Meanwhile, value regularization promotes conservative value functions to alleviate OOD overestimation via establishing precise lower bounds for Q-values to facilitate in-sample action selection (Lyu et al., 2022; Park et al., 2024). Model-based approaches construct task-oriented world models from offline data, including one-step dynamics (Yu et al., 2020; Chemingui et al., 2024) and multi-step sequence models (Hafner et al., 2020; 2023), which are subsequently used for offline reward penalization (Yu et al., 2021; Kidambi et al., 2020) and online decision planning to ensure in-distribution actions (Hansen et al., 2022; 2023).

Return-conditioned supervised learning trains trajectory generators conditioned on returns, with transformers (Chen et al., 2021; Schmied et al., 2024; Kong et al., 2025) and diffusion models (Janner et al., 2022; Ajay et al., 2022) as common backbones. Despite their effectiveness, these approaches are fundamentally limited by reliance on the dataset distribution.

Data Augmentation for Offline RL synthesizes additional interactions to enrich the dataset for subsequent RL training. Early work employs dynamics ensembles to generate transitions with uncertainty weighting (Zhang et al., 2023) or diffusion models for flexible one-step oversampling (Lu et al., 2023). More recent efforts shift toward trajectory-level generation. Some work focuses on on-policy rollouts, guided either by policy outputs (Jackson et al., 2024), return value (Lee et al., 2024), or interaction histories (He et al., 2023). Other approaches explore prioritized augmentation based on dynamics error or curiosity (Wang et al., 2024), as well as trajectory recomposition through reverse generation (Yang & Wang, 2025) or diffusion-based stitching (Li et al., 2024). Existing data augmentation methods for offline RL typically rely on a single generative process within a single forward-future or backward-history rollout paradigm, which preserves the original trajectory-level connectivity of the dataset. In contrast, BiTrajDiff synthesizes both forward and backward trajectories conditioned on shared intermediate states, allowing it to bridge distinct behavior modes among datasets and recover global connectivity, resulting in substantial improvements in offline RL performance.

3. Preliminary

RL Paradigm (Sutton et al., 1998) is formally defined as a Markov Decision Process (MDP), $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0 \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ represents the environment dynamics, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, $\gamma \in [0, 1]$ is the discount factor, and ρ_0 denotes the initial state distribution. At each timestep t , an agent executes an action a_t according to its policy $\pi(\cdot|s_t)$ in the current state s_t , transitioning to the subsequent state s_{t+1} with reward r_t based on P . The objective of the agent is to find an optimal policy π^* that maximizes the expected discounted return: $\pi^* = \arg \max_{\pi} \mathcal{J}(\pi) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$. In contrast to online RL, offline RL (Levine et al., 2020) relies solely on a static trajectories dataset $\mathcal{D} = \{\tau^i\}_{i=1}^N$, where N is the number of trajectories and each trajectory $\tau^i = \{(s_t^i, a_t^i, r_t^i)\}_{t=0}^{T-1}$ comprises a sequence of state-action-reward tuples.

Diffusion Models (Ho et al., 2020; Song et al., 2021) are generative frameworks that learn data distributions through a forward diffusion process and a reverse denoising process. The forward process gradually corrupts data $\mathbf{x}_0$ via a pre-

defined noise schedule $\{\beta_k\}_{k=1}^K$, where each transition is defined as $q(\mathbf{x}_k|\mathbf{x}_{k-1}) = \mathcal{N}(\mathbf{x}_k; \sqrt{1 - \beta_k}\mathbf{x}_{k-1}, \beta_k\mathbf{I})$. As $K \rightarrow \infty$, the terminal distribution $q(\mathbf{x}_K)$ converges to an isotropic Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$. The reverse process, parameterized by θ , aims to iteratively reconstruct the original data through learnable Gaussian transitions: $p_{\theta}(\mathbf{x}_{k-1}|\mathbf{x}_k) = \mathcal{N}(\mathbf{x}_{k-1}; \mu_{\theta}(\mathbf{x}_k, k), \Sigma_{\theta}(\mathbf{x}_k, k))$. By maximizing the empirical lower bound (Sohn et al., 2015) on the log-likelihood of the sampled data, the diffusion model can be trained with a simplified surrogate loss (Sohn et al., 2015):

$$\mathcal{L}_{\text{denoise}}(\theta) = \mathbb{E}_{\mathbf{x}_0 \sim q, k \sim \mathcal{U}\{1, K\}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\|\epsilon_{\theta}(\mathbf{x}_k, k) - \epsilon\|_2^2 \right], \quad (1)$$

where $\mathcal{U}$ denotes the discrete uniform distribution, and ϵ_{θ} is the deep model parameterized with θ to predict the noise. Both the mean $\mu_{\theta}(\mathbf{x}_k, k)$ and covariance $\Sigma_{\theta}(\mathbf{x}_k, k)$ of the reverse process are analytically derivable from ϵ_{θ} . The sampling is then performed by initializing with $\mathbf{x}_K \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and iteratively applying the learned reverse transitions.

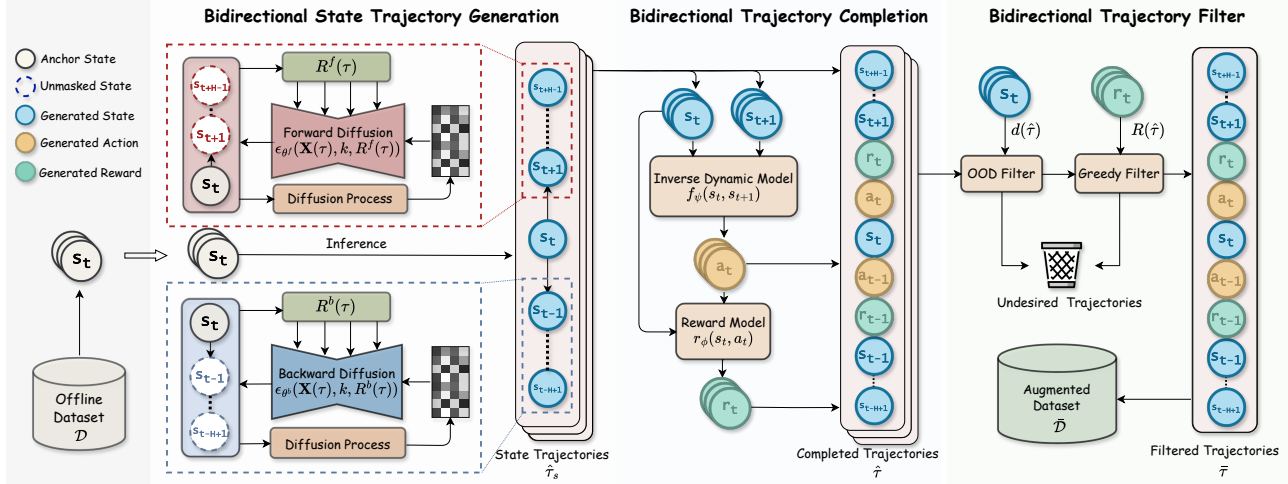
4. Methodology

This section introduces the BiTrajDiff framework for offline RL, which comprises two core components: *bidirectional diffusion training* and *bidirectional trajectory generation*. During the *bidirectional diffusion training* phase, two distinct diffusion models are employed to model the distributions of forward-future and backward-history state sequences within trajectory data. Conditioned on an intermediate state and cumulative reward signals, these models generate state sequences in corresponding temporal directions. Then, in the subsequent *bidirectional trajectory generation* phase, the generated forward and backward trajectories are reconciled through filling and filtering operations to produce novel global trajectories that connect previously unreachable states in the original dataset. The framework of our method is presented in Figure 2.

4.1. Bidirectional Diffusion Training

Building upon recent advances in diffusion-based sequence generation (Fathi et al., 2025; Gong et al., 2022), we propose a bidirectional trajectory generation framework comprising independent forward and backward trajectory diffusion models. These models are separately trained on offline datasets to learn the trajectory distribution of the behavior policy, facilitating bidirectional state trajectory generation from shared intermediate ‘‘anchor’’ states.

Formally, for each diffusion model, the state trajectory is generated simultaneously per time step t over the planning horizon H : $\mathbf{x}_k(\tau) = \{s_t, s_{t+1}, \dots, s_{t+H-1}\}_k$, where k denotes the denoise timestep and t represents the MDP timestep. In this way, the conditional generative problem


 Figure 2. An illustrative diagram of our *Bidirectional Trajectory Diffusion* (BiTrajDiff) method.

via the diffusion model can be formulated as:

$$\min_{\theta} \mathcal{L}(\theta) = \mathbb{E}_{\tau \sim \mathcal{D}} [-\log p_{\theta}(\mathbf{x}_0(\tau) | \mathbf{y}(\tau))], \quad (2)$$

where $\mathbf{x}_0(\tau)$ denotes the final generated subtrajectory of diffusion, $\mathbf{y}(\tau)$ represents the condition for generation, and θ represents the learnable parameters of the diffusion model. While various condition metrics are available, such as temporal difference error, Q-function (Wang et al., 2024), our framework utilizes the cumulative return R as conditional input $\mathbf{y}(\tau)$. This approach, consistent with prior works (Ajay et al., 2022; Ding et al., 2024), enables the generation of trajectories with desired returns starting from the specific states. However, to enable bidirectional trajectory generation, the formations of the backward and forward diffusion condition inputs are quite distinct.

Specifically, the forward diffusion model generates finite-horizon state trajectories $\mathbf{x}_0^f(\tau) = \{s_t, s_{t+1}, \dots, s_{t+H-1}\}$ **started at initial state s_t with target return $R^f(\tau) = \sum_{i=t}^{\infty} \gamma^{i-t} r_i$** , thus generating physically consistent future trajectories within the offline distribution. Conversely, the backward diffusion model reconstructs historical trajectories $\mathbf{x}_0^b(\tau) = \{s_{t-H+1}, \dots, s_t\}$ that **ends on terminal state s_t and accrued return $R^b(\tau) = \sum_{i=0}^t \gamma^{t-i} r_i$** . For brevity and clarity, we utilize superscript-free notation (e.g., $\mathbf{x}_k$, $\mathbf{y}$, $R(\tau)$) in the following sections to denote computational processes common to both directional models.

Based on the two distinct conditions, we can model the conditional trajectory generation processes by the diffusion sampling composed of the forward noising process $q(\mathbf{x}_{k+1}(\tau) | \mathbf{x}_k(\tau))$ and the reverse denoising process $p_{\theta}(\mathbf{x}_{k-1}(\tau) | \mathbf{x}_k(\tau), \mathbf{y}(\tau))$ constructed with the diffusion denoise model ϵ_{θ} . Meanwhile, the forward and backward diffusion models ϵ_{θ} in our BiTrajDiff framework are parameterized with θ^f and θ^b , respectively. Thus, given the

condition $\mathbf{y}(\tau)$, they are able to generate the denoised trajectory $\mathbf{x}_0(\tau)$. This process begins with sampling random noise $\mathbf{x}_K \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, followed by iterative denoising steps where $\mathbf{x}_{k-1} \sim p_{\theta}$, for $k = \{K, \dots, 1\}$.

To realize conditional diffusion model training and inference, we employ the Classifier-Free Guidance (CFG) (Ho & Salimans, 2022), which uses a single denoise model to handle both conditional and unconditional generation. Formally, the perturbed noise after CFG can be formed as:

$$\hat{\epsilon} = \omega * \epsilon_{\theta}(\mathbf{x}_k(\tau), k, \mathbf{y}(\tau)) + (1 - \omega) * \epsilon_{\theta}(\mathbf{x}_k(\tau), k, \emptyset), \quad (3)$$

where the scalar $\omega \in [0, 1]$ denotes the guidance weight of CFG. Setting ω to 0 degrades the generation to unconditional generation. Conversely, an increased value of ω promotes a stronger adherence of the generated samples to the provided condition $\mathbf{y}(\tau)$. During training, the diffusion denoisers are trained with an objective that enables them to predict noise conditioned on $\mathbf{y}(\tau)$ and also unconditionally. This is typically achieved by randomly dropping the condition $\mathbf{y}(\tau)$ during training with probability p . Specifically, we have the following loss function:

$$\mathcal{L}_{\text{denoise}}(\theta) = \mathbb{E}_{\tau \sim \mathcal{D}, k \sim \mathcal{U}\{1, K\}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \beta \sim \text{Bern}(p)} [\|\epsilon - \epsilon_{\theta}(\mathbf{x}_k(\tau), k, (1 - \beta)\mathbf{y}(\tau) + \beta\emptyset)\|^2], \quad (4)$$

where $\text{Bern}(p)$ represents the Bernoulli distribution. In our implementation, we fix the state condition s_t in $\mathbf{x}_k(\tau)$ during the adding-noise training as well as the denoising inference stage for simplicity and efficiency following (Ajay et al., 2022; Yang & Wang, 2025), while the diffusion denoise models ϵ_{θ} only take the cumulative return $R(\tau)$ components of $\mathbf{y}(\tau)$ as condition inputs.

4.2. Bidirectional Trajectory Generation

Based on the pretrained bidirectional diffusion model, we introduce a novel DA pipeline through three key stages: bidirectional state trajectory generation, bidirectional trajectory completion, and bidirectional trajectory filtering. The pipeline synthesizes global trajectories, thereby establishing connections between states not accessible in the datasets. This process extends behavioral coverage while upholding physical consistency, ultimately producing effective augmented data for offline RL algorithms training.

4.2.1. BIDIRECTIONAL TRAJECTORY GENERATION

We utilize the pretrained forward and backward trajectory diffusion model in section 4.1 to generate bidirectional state trajectories. Prior methods (Lu et al., 2023; Li & Zhang, 2024) model only the conditional distribution of trajectories originating from a given state, neglecting the historical pathways leading to those states. As a result, these data augmentation techniques yield only limited expansion in behavior diversity. To address this limitation without introducing additional noise, our proposed method, BiTrajDiff, generates both forward and backward trajectories and integrates them by stitching at a shared intermediate state s_t , which serves as a consistent anchor.

Specifically, we sample the candidate state s from the offline dataset $\mathcal{D}$ as the condition state s_t . By combining the state s_t with the manually selected return signal $R(\tau)$ into the $\mathbf{y}(\tau)$, we can leverage the pre-trained bidirectional diffusion model θ to generate two distinct trajectories: a forward-future trajectory $\mathbf{x}_0^f(\tau)$ originates from s_t , and a backward-history trajectory $\mathbf{x}_0^b(\tau)$ concludes at s_t . Since s_t is both the start state of $\mathbf{x}_0^f(\tau)$ and the end state of $\mathbf{x}_0^b(\tau)$, we can directly stitch them to construct the bidirectional state trajectory $\hat{\tau}_s(s_t)$. Formally, let $\mathbf{x}_0(\tau)[i:j]$ denote the observation slice from time i to time j , the stitched bidirectional state trajectory $\hat{\tau}_s(s_t)$ can be formulated as:

$$\begin{aligned} \hat{\tau}_s(s_t) &= \left\{ \mathbf{x}_0^b(\tau)[0:H-1], s_t, \mathbf{x}_0^f(\tau)[1:H] \right\} \\ &= \{ \hat{s}_{t-H+1}, \dots, \hat{s}_{t-1}, s_t, \hat{s}_{t+1}, \dots, \hat{s}_{t+H-1} \}, \end{aligned} \quad (5)$$

where H is the horizon, and $\hat{s}_i$ is the diffusion generated states at MDP timestep i . By conditioning on and concatenating at the intermediate anchor state s_t , we construct a state trajectory, $\hat{\tau}_s(s_t)$, that connects $\hat{s}_{t-H+1}$ and $\hat{s}_{t+H-1}$. This process enables the connection of states that may be disconnected in the original dataset, thereby substantially enriching the diversity of observed behavioral patterns. Appendix K provides a formal interpretation of the trajectory-level connectivity gap between our bidirectional paradigm and existing single-direction paradigms, and further explains why bidirectional stitching is able to recover couplings that cannot be captured by single-direction trajectory rollouts.

4.2.2. BIDIRECTIONAL TRAJECTORY COMPLETION

In this completion process, the generated bidirectional state trajectories $\hat{\tau}_s(s_t)$ are further completed by adding the action and reward signal. We introduce the Inverse Dynamics Model (IDM): $f_\psi(s_t, s_{t+1}) : \mathcal{S} \times \mathcal{S} \rightarrow \mathcal{A}$ and the Reward Model (RM): $r_\phi(s_t, a_t) : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. The supervised training objective of the two models is:

$$\mathcal{L}(\psi, \phi) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{D}} \left[\|f_\psi(s_t, s_{t+1}) - a_t\|^2 + \|r_\phi(s_t, a_t) - r_t\|^2 \right]. \quad (6)$$

Thus, by feeding the generated state trajectory $\hat{\tau}_s(s_t)$ to the trained IDM and RM sequentially, we obtain the completed bidirectional trajectory : $\hat{\tau}(s_t) = \{(\hat{s}_i, \hat{a}_i, \hat{r}_i, \hat{s}_{i+1})\}_{i=t-H+1}^{t+H-2}$, where $\hat{a}_i = f_\psi(\hat{s}_i, \hat{s}_{i+1})$ and $\hat{r}_i = r_\phi(\hat{s}_i, \hat{a}_i)$. Therefore, we can obtain a generated dataset $\hat{\mathcal{D}} = \{\hat{\tau}(s_{t_i})\}_{i=1}^N$ with N generated trajectories.

4.2.3. BIDIRECTIONAL TRAJECTORY FILTER

To ensure dataset quality, we employ a two-stage filtering mechanism that removes both out-of-distribution and sub-optimal trajectories. This selective process preserves the reliability of the augmented data without sacrificing diversity. In BiTrajDiff, the trajectory filter comprises an OOD trajectory filter and a greedy trajectory filter.

OOD Trajectory Filter. We model the OOD trajectory filter as a novelty detection model (Pimentel et al., 2014), which addresses the identification of data instances that exhibit a significant deviation from the offline training dataset. And we utilize the Isolation Forest model (Liu et al., 2008) for OOD transition detection, which assigns an anomaly score to each data instance, reflecting its susceptibility to isolation with shorter path lengths in randomly partitioned trees. Specifically, we construct the isolation forest on the original dataset $\mathcal{D}$, which is able to give each observation $\hat{s}_i$ an anomaly score $d(\hat{s}_i)$. We defined the OOD score of a generated trajectory $d(\hat{\tau}) = \sum_{i=t-H+1}^{t+H-2} d(\hat{s}_i)$. Then we sort $\hat{\mathcal{D}}$ by $d(\hat{\tau})$ and retain the top- C_{ood} trajectories with the smallest values, where C_{ood} is a hyperparameter.

Greedy Trajectory Filter. The greedy trajectory filter selects the generated trajectories with the highest cumulative reward. Specifically, we sort the C_{ood} trajectories retained from OOD trajectory filter by the sum reward $R(\hat{\tau}) = \sum_{i=t-H+1}^{t+H-2} \hat{r}_i$. The top- C_{greedy} trajectories with the highest sum rewards are picked as the final generated trajectories dataset $\hat{\mathcal{D}}$ for offline RL training.

In summary, BiTrajDiff operates in two key stages. Initially, it trains a bidirectional diffusion model to capture the distributions of forward-future and backward-history trajectories.

Table 1. Performance of our BiTrajDiff and baselines on the locomotion tasks. $\pm$ corresponds to the standard deviation of the performance on 5 random seeds. The best and the second-best results of each setting are marked as **bold** and underline, respectively. Detailed reports about CQL and DT are reported in Appendix C.

Source	Task	IQL (Kostrikov et al., 2021)					TD3BC (Fujimoto & Gu, 2021)				
		Base	RTDiff	Synther	DiffStitch	Ours	Base	RTDiff	Synther	DiffStitch	Ours
medium	halfcheetah	48.2 $\pm$ 0.2	48.9 $\pm$ 0.2	49.2 $\pm$ 0.2	48.6 $\pm$ 0.2	48.6 $\pm$ 0.2	48.4 $\pm$ 0.2	49.4 $\pm$ 0.2	49.7 $\pm$ 0.4	<u>50.1</u> $\pm$ 0.6	50.3 $\pm$ 0.4
	hopper	67.0 $\pm$ 1.9	62.0 $\pm$ 2.4	61.1 $\pm$ 2.7	65.5 $\pm$ 4.7	81.1 $\pm$ 5.0	60.4 $\pm$ 3.7	62.6 $\pm$ 5.1	61.2 $\pm$ 6.1	<u>67.9</u> $\pm$ 3.7	79.0 $\pm$ 3.5
	walker2d	77.6 $\pm$ 5.3	82.7 $\pm$ 4.0	<u>85.8</u> $\pm$ 0.7	79.8 $\pm$ 1.8	86.7 $\pm$ 1.7	82.0 $\pm$ 2.7	85.4 $\pm$ 1.1	83.4 $\pm$ 2.0	<u>85.5</u> $\pm$ 0.7	86.7 $\pm$ 1.0
medium expert	halfcheetah	90.1 $\pm$ 4.4	92.0 $\pm$ 3.1	86.6 $\pm$ 6.2	90.3 $\pm$ 8.0	95.3 $\pm$ 0.1	92.3 $\pm$ 5.1	93.9 $\pm$ 2.1	95.5 $\pm$ 0.8	<u>95.9</u> $\pm$ 0.7	96.3 $\pm$ 0.4
	hopper	105.4 $\pm$ 6.3	110.5 $\pm$ 0.5	111.2 $\pm$ 0.7	108.6 $\pm$ 1.2	<u>110.9</u> $\pm$ 0.6	94.2 $\pm$ 11.1	<u>106.8</u> $\pm$ 7.0	106.2 $\pm$ 6.7	97.5 $\pm$ 9.9	109.0 $\pm$ 5.1
	walker2d	<u>112.4</u> $\pm$ 0.4	111.9 $\pm$ 0.5	112.1 $\pm$ 0.6	111.2 $\pm$ 0.3	112.5 $\pm$ 0.5	109.3 $\pm$ 0.2	<u>109.6</u> $\pm$ 0.4	109.4 $\pm$ 0.0	<u>109.6</u> $\pm$ 0.5	110.2 $\pm$ 0.3
medium replay	halfcheetah	43.5 $\pm$ 0.2	43.7 $\pm$ 0.8	46.2 $\pm$ 0.3	43.6 $\pm$ 0.1	44.1 $\pm$ 0.2	44.1 $\pm$ 0.1	44.0 $\pm$ 0.3	44.5 $\pm$ 0.6	45.1 $\pm$ 0.5	45.0 $\pm$ 0.6
	hopper	94.7 $\pm$ 4.0	97.7 $\pm$ 2.7	<u>100.6</u> $\pm$ 0.4	96.4 $\pm$ 5.6	102.8 $\pm$ 0.6	64.4 $\pm$ 8.9	72.1 $\pm$ 18.9	<u>76.4</u> $\pm$ 3.0	71.5 $\pm$ 21.0	85.0 $\pm$ 10.3
	walker2d	73.3 $\pm$ 7.5	74.0 $\pm$ 5.2	<u>86.7</u> $\pm$ 1.1	74.9 $\pm$ 8.2	87.6 $\pm$ 2.2	80.9 $\pm$ 6.0	<u>83.5</u> $\pm$ 7.4	81.1 $\pm$ 5.9	80.5 $\pm$ 5.2	89.9 $\pm$ 1.4
Average		79.1	80.4	82.2	79.9	85.5	75.1	78.6	78.6	78.2	83.5

Table 2. Performance of our BiTrajDiff and baselines on the navigation and manipulation tasks.

Task	IQL (Kostrikov et al., 2021)					TD3BC (Fujimoto & Gu, 2021)				
	Base	RTDiff	Synther	DiffStitch	Ours	Base	RTDiff	Synther	DiffStitch	Ours
maze2d-umaze	56.0 $\pm$ 6.1	53.1 $\pm$ 2.2	51.9 $\pm$ 4.8	52.1 $\pm$ 5.9	62.3 $\pm$ 2.7	38.1 $\pm$ 10.2	<u>43.1</u> $\pm$ 6.4	40.3 $\pm$ 10.0	42.5 $\pm$ 8.3	45.3 $\pm$ 3.1
maze2d-medium	41.2 $\pm$ 2.9	50.8 $\pm$ 9.6	66.9 $\pm$ 16.9	46.4 $\pm$ 10.2	72.2 $\pm$ 14.8	30.2 $\pm$ 16.3	<u>36.3</u> $\pm$ 9.9	34.8 $\pm$ 14.2	34.7 $\pm$ 3.0	45.7 $\pm$ 7.6
maze2d-large	67.9 $\pm$ 4.2	74.3 $\pm$ 5.2	70.9 $\pm$ 2.5	64.2 $\pm$ 6.1	<u>71.3</u> $\pm$ 2.3	95.3 $\pm$ 22.7	102.8 $\pm$ 8.9	100.6 $\pm$ 34.3	<u>106.1</u> $\pm$ 11.4	129.3 $\pm$ 26.0
antmaze-umaze-diverse	53.0 $\pm$ 4.7	61.6 $\pm$ 5.7	55.4 $\pm$ 14.2	55.4 $\pm$ 5.4	63.4 $\pm$ 7.9	43.2 $\pm$ 7.4	45.8 $\pm$ 4.4	45.0 $\pm$ 10.3	49.4 $\pm$ 4.8	<u>47.0</u> $\pm$ 2.1
antmaze-medium-diverse	71.2 $\pm$ 5.0	73.2 $\pm$ 8.8	81.2 $\pm$ 6.4	69.2 $\pm$ 17.2	86.2 $\pm$ 5.8	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	6.4 $\pm$ 5.9	0.0 $\pm$ 0.0	4.8 $\pm$ 1.7
antmaze-large-diverse	18.0 $\pm$ 8.4	53.2 $\pm$ 9.5	<u>57.8</u> $\pm$ 6.4	12.8 $\pm$ 15.2	63.0 $\pm$ 9.3	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	<u>0.4</u> $\pm$ 0.5	0.0 $\pm$ 0.0	1.2 $\pm$ 0.9
Maze Average	51.2	61.0	64.0	50.0	69.7	34.5	38.0	37.9	38.8	45.6
kitchen-complete	43.3 $\pm$ 16.0	<u>51.9</u> $\pm$ 7.4	42.8 $\pm$ 16.4	38.3 $\pm$ 8.9	59.1 $\pm$ 12.3	0.1 $\pm$ 0.1	0.8 $\pm$ 0.5	0.1 $\pm$ 0.2	<u>0.5</u> $\pm$ 0.7	1.5 $\pm$ 1.0
kitchen-partial	68.0 $\pm$ 5.1	<u>69.8</u> $\pm$ 2.9	70.3 $\pm$ 2.8	62.9 $\pm$ 9.4	69.7 $\pm$ 1.8	11.8 $\pm$ 9.7	8.9 $\pm$ 7.0	10.9 $\pm$ 1.8	13.0 $\pm$ 9.3	<u>13.0</u> $\pm$ 5.9
kitchen-mixed	60.7 $\pm$ 2.5	64.4 $\pm$ 6.1	62.6 $\pm$ 2.7	<u>63.4</u> $\pm$ 5.2	62.9 $\pm$ 4.9	8.7 $\pm$ 5.1	7.7 $\pm$ 0.7	<u>10.7</u> $\pm$ 5.0	6.0 $\pm$ 4.7	30.9 $\pm$ 12.6
Kitchen Average	57.3	62.0	58.6	54.9	63.9	6.9	5.8	7.2	6.5	15.1

Subsequently, BiTrajDiff generates bidirectional trajectories conditioned on anchor states sampled from $\mathcal{D}$. These generated sequences undergo further completion and filtering to create a high-fidelity augmented dataset $\tilde{\mathcal{D}}$, which is then mixed with $\mathcal{D}$ to train downstream offline RL algorithms. We adopt CleanDiffuser (Dong et al., 2024) as the backbone for BiTrajDiff, owing to its generality and robustness. The pseudocode of BiTrajDiff is presented in Appendix A.

5. Experiments

To demonstrate the effectiveness of the proposed BiTrajDiff, we conduct experiments on the D4RL benchmark (Fu et al., 2020). Our evaluation seeks to answer the following questions: (1) Does BiTrajDiff outperform existing DA methods across various offline RL algorithms? (Section 5.2 and Appendix C-D). (2) How do different components of BiTrajDiff affect the offline RL performance? (Section 5.3 and Appendix E-G). (3) Can BiTrajDiff find the potentially valuable state leading to high return more effectively than other DA baselines? (Section 5.4 and Appendix H). (4) Can

BiTrajDiff generate more diverse trajectories compared to single-direction DA approaches? (Section 5.5)

5.1. Experiment Settings

We evaluate our approach on the D4RL benchmark (Fu et al., 2020). We compare BiTrajDiff with three classical DA methods: Synther (Lu et al., 2023), DiffStitch (Li et al., 2024) and RTDiff (Yang & Wang, 2025). We select four representative offline RL algorithms as baselines: IQL (Kostrikov et al., 2021), TD3BC (Fujimoto & Gu, 2021), CQL (Kumar et al., 2020), and DT (Chen et al., 2021). These methods have been widely adopted as standard baselines in offline RL due to their stable performance. The hyperparameter setting can be viewed in Appendix B.2.

5.2. Main Results

Results for Locomotion Tasks. The experimental results on D4RL locomotion tasks are summarized in Table 1. Our proposed BiTrajDiff demonstrates consistent superiority in

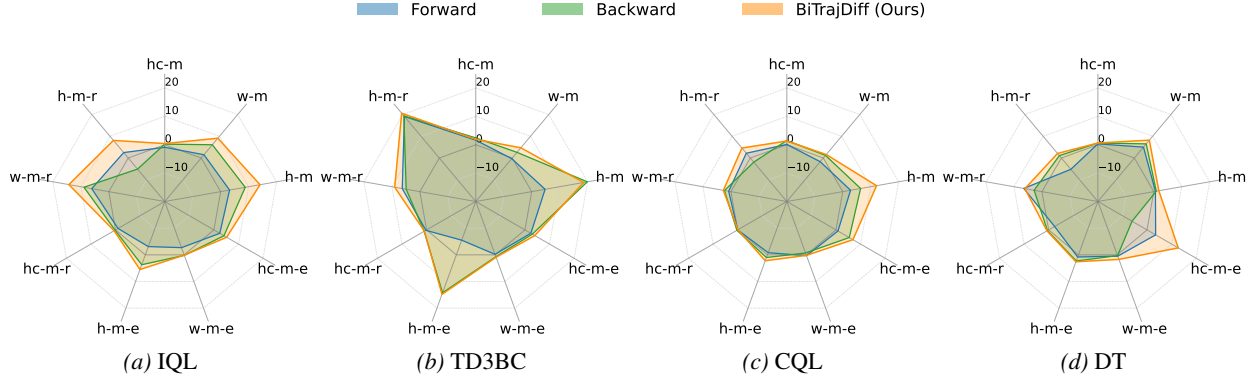


Figure 3. Performance improvement comparison of offline RL algorithms augmented with single- and bi-directional diffusion trajectories. The task abbreviations are listed in Table S1.

various task settings. Meanwhile, results on CQL and DT, presented in Appendix C, exhibit a similar phenomenon. These consistent gains across the offline RL algorithms underscore the effectiveness of BiTrajDiff.

Results for Maze and Franka Kitchen Tasks. Table 2 reports the experimental results on the D4RL Maze and Franka Kitchen tasks. BiTrajDiff achieves the highest average return under both the IQL and TD3BC algorithms. Since the Maze and Franka Kitchen tasks are sparse-reward settings, which are typically detrimental to RL training, the availability of high-quality data is particularly critical. BiTrajDiff tackles this challenge by enhancing data diversity quality, yielding superior and more stable performance.

5.3. Ablation Analysis

Ablation Study of the Direction of Diffusion Generation.

To confirm the effectiveness of our bidirectional generation paradigm, we compare the performance improvement with the augmented diffusion trajectories generated in a single direction and in a bidirectional way, as shown in Figure 3. While the forward method yields moderate improvements, it is consistently outperformed because it neglects the historical transitions leading to critical, high-reward states. Conversely, the backward method achieves substantial gains by exploring trajectories toward a predefined initial state, thus mitigating the overestimation risk associated with unknown states (Yang & Wang, 2025). Finally, demonstrating consistently superior and robust performance, our bi-directional framework, BiTrajDiff, empirically validates that by simultaneously synthesizing forward-future and backward-history trajectories, BiTrajDiff constructs novel, high-quality paths absent from the original dataset.

Ablation Study of Trajectory Filters. We compare the IQL and TD3BC training process augmented by BiTrajDiff datasets with different trajectory filter strategies. The learning curves are presented in Figure 4. Without any filters, the training process is highly volatile, as unfiltered, potentially

OOD trajectories introduce suboptimal or even illegal states and cause erratic policy updates. In contrast, applying the OOD filter substantially stabilizes the training process by constraining synthetic data to a plausible behavioral space, yet this configuration consistently underperforms the full BiTrajDiff model. This is because the greedy filter selectively retains trajectories that yield higher returns, thereby actively guiding the policy toward good behaviors and facilitating the discovery of better behavior patterns.

Ablation Study of Augmented Data Ratio. We investigate the balance between original and augmented data by varying the ratio σ of augmented to original data, with the results presented in Figure 5. When σ is too low, performance improvements remain limited across the tested algorithms, illustrating that a limited amount of augmented data is not enough to provide offline RL algorithms with new behavior patterns. Conversely, increasing the ratio to a range of 30% ~ 50% leads to a substantial performance boost and stability. However, when the ratio is increased to 100%, the performance becomes highly unstable, which suggests that the excessive noise introduced by an overabundance of synthetic data hinders offline RL algorithms from learning effectively. Finally, we select $\sigma = 30\%$ as the fixed hyperparameter due to its stable performance improvement.

5.4. Effectiveness under n -step TD Estimator

We employ the n -step TD estimator to update the critic networks in IQL and TD3BC under the original dataset and the DA-augmented data, and further compare performance across different values of n as shown in Figure 6. The n -step TD estimator (De Asis et al., 2018) enables the discovery of high-return solutions by looking ahead from more distant states, making performance improvement a natural indicator of the quality of DA-generated datasets. As shown in Figure 6, all three DA methods consistently outperform the base method, suggesting that the generated trajectories effectively uncover more rewarding states. How-

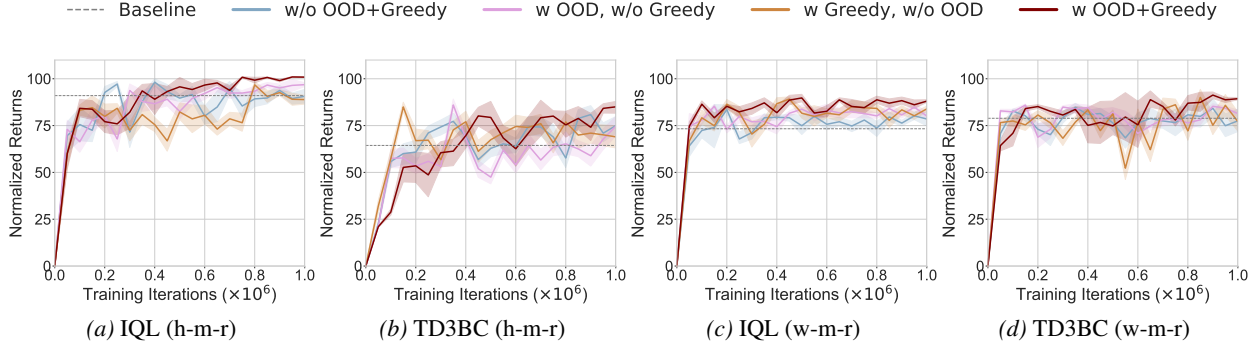


Figure 4. Learning curves of BiTrajDiff with different trajectory filter strategies. Detailed results are reported in Appendix F.

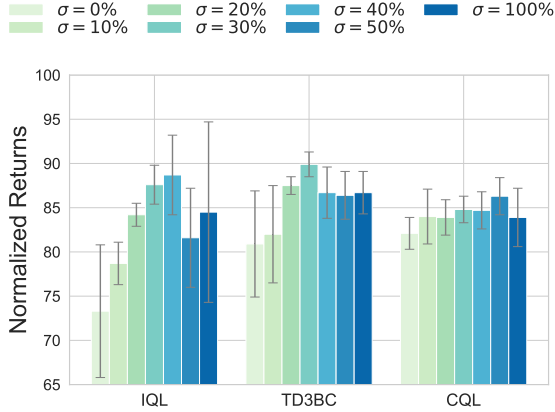


Figure 5. Compare the returns of BiTrajDiff with different augmented data ratios σ in the walker2d-medium-replay task. Detailed results are reported in Appendix G.

ever, as n increases, the performance inevitably declines due to the growing variance in value estimation. Meanwhile, for DA methods, this effect is compounded by the accumulated mismatching errors introduced by the inverse dynamics model, which negatively impact TD estimation. In particular, our method consistently achieves the highest performance across all tested settings and maintains its advantage over RTDiff and Diffstitch as the n -step horizon increases. These results verify the effectiveness of BiTrajDiff for offline RL algorithms under n -step TD estimator. By stitching forward-future and backward-history trajectories, BiTrajDiff obtains a superior capability to synthesize high-quality trajectories compared to existing DA methods.

5.5. Visualization

To evaluate the accuracy and diversity robustness of the BiTrajDiff framework, we compare its generated trajectories with those produced by single-direction forward and backward diffusion models. Following Lu et al. (2023), we employ two quantitative metrics to assess a generated trajectory $\hat{\tau}$: (1) Dynamic Error: $\mathcal{E}_{\text{Dyn}}(\hat{\tau}) = \sum_t \|\hat{s}_{t+1} - s_{t+1}\|_2^2$, which measures trajectory accuracy by summing

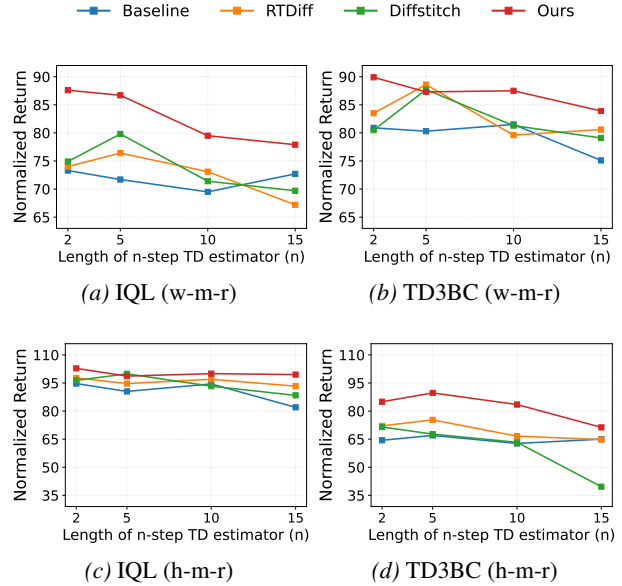


Figure 6. Comparisons of different methods under varying n -step TD estimators. Detailed results are reported in Appendix H.

the mean squared error between the predicted next states $\hat{s}_{t+1}$ and the ground truth s_{t+1} ; (2) L2 Distance: $\mathcal{E}_{\text{L2D}}(\hat{\tau}) = \min_{\tau \in \mathcal{D}} \sum_t \|\hat{s}_t - s_t\|_2$, defined as the minimal L2 distance between the generated trajectory and any trajectory of equal length in $\mathcal{D}$, capturing trajectory diversity. As shown in Figure 7, while forward and backward diffusion models achieve low $\mathcal{E}_{\text{Dyn}}$, they exhibit limited $\mathcal{E}_{\text{L2D}}$, indicating that the generated trajectories closely resemble those in the original dataset and thus offer limited performance gains. In contrast, BiTrajDiff demonstrate significantly broader marginal distributions in $\mathcal{E}_{\text{L2D}}$ while maintaining competitive $\mathcal{E}_{\text{Dyn}}$, reflecting enhanced behavioral diversity without sacrificing dynamic consistency.

6. Conclusion

In this paper, we present BiTrajDiff, a novel diffusion-based framework that enhances offline RL by enabling bidirec-

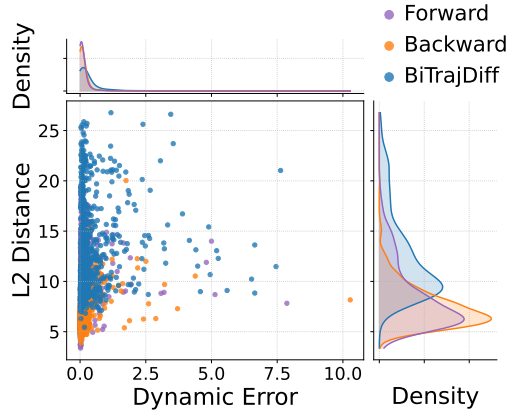


Figure 7. BiTrajDiff vs forward/backward diffusion: dynamic error and L2 distance: Comparisons of dynamic error and L2 distance.

tional trajectory generation from shared intermediate anchor states. Unlike conventional single rollout paradigms, which model forward or backward trajectories from observed states, BiTrajDiff generates both forward-future and backward-history trajectories, conditioned on shared intermediate anchor states, effectively bridging previously disconnected behavior patterns in the state space. This leads to significantly more diverse and effective trajectories, improving offline RL performance beyond the capability of other DA frameworks. Extensive experiments on the D4RL benchmark also demonstrate that BiTrajDiff significantly outperforms existing data augmentation baselines across diverse offline RL algorithms. Future work will explore bidirectional generation for offline RL under multi-task settings with broader behavioral patterns.

Impact Statement

This paper presents work whose goal is to advance the field of offline reinforcement learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Ajay, A., Du, Y., Gupta, A., Tenenbaum, J., Jaakkola, T., and Agrawal, P. Is conditional generative modeling all you need for decision-making? *arXiv preprint arXiv:2211.15657*, 2022.
- Chemingui, Y., Deshwali, A., Hoang, T. N., and Doppa, J. R. Offline model-based optimization via policy-guided gradient search. In *AAAI Conference on Artificial Intelligence*, 2024.
- Chen, K., Luo, W., Liu, S., Wei, Y., Zhou, Y., Qing, Y., Zhang, Q., Song, J., and Song, M. Powerformer: A section-adaptive transformer for power flow adjustment. *arXiv preprint arXiv:2401.02771*, 2024.
- Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., and Mordatch, I. Decision transformer: Reinforcement learning via sequence modeling. In *Advance in Neural Information Processing Systems*, 2021.
- Chen, X., Ghadirzadeh, A., Yu, T., Wang, J., Gao, A. Y., Li, W., Bin, L., Finn, C., and Zhang, C. Lapo: Latent-variable advantage-weighted policy optimization for offline reinforcement learning. In *Advance in Neural Information Processing Systems*, 2022.
- De Asis, K., Hernandez-Garcia, J., Holland, G., and Sutton, R. Multi-step reinforcement learning: A unifying algorithm. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Ding, Z., Zhang, A., Tian, Y., and Zheng, Q. Diffusion world model: Future modeling beyond step-by-step rollout for offline reinforcement learning. *arXiv preprint arXiv:2402.03570*, 2024.
- Dong, Z., Yuan, Y., Hao, J., Ni, F., Ma, Y., Li, P., and Zheng, Y. Cleandiffuser: An easy-to-use modularized library for diffusion models in decision making. *arXiv preprint arXiv:2406.09509*, 2024.
- Fathi, N., Scholak, T., and Noël, P.-A. Unifying autoregressive and diffusion-based sequence generation. *arXiv preprint arXiv:2504.06416*, 2025.
- Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- Fujimoto, S. and Gu, S. S. A minimalist approach to offline reinforcement learning. *Advances in neural information processing systems*, 34:20132–20145, 2021.
- Fujimoto, S., Meger, D., and Precup, D. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, 2019.
- Gong, S., Li, M., Feng, J., Wu, Z., and Kong, L. Diffuseq: Sequence to sequence text generation with diffusion models. *arXiv preprint arXiv:2210.08933*, 2022.
- Hafner, D., Lillicrap, T., Norouzi, M., and Ba, J. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.
- Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

- Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.
- Hansen, N., Su, H., and Wang, X. Td-mpc2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.
- He, H., Bai, C., Xu, K., Yang, Z., Zhang, W., Wang, D., Zhao, B., and Li, X. Diffusion model is an effective planner and data synthesizer for multi-task reinforcement learning. In *Advance in Neural Information Processing Systems*, 2023.
- Ho, J. and Salimans, T. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020.
- Jackson, M. T., Matthews, M. T., Lu, C., Ellis, B., Whiteson, S., and Foerster, J. Policy-guided diffusion. *arXiv preprint arXiv:2404.06356*, 2024.
- Janner, M., Du, Y., Tenenbaum, J. B., and Levine, S. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.
- Kidambi, R., Rajeswaran, A., Netrapalli, P., and Joachims, T. Morel: Model-based offline reinforcement learning. In *Advance in Neural Information Processing Systems*, 2020.
- Kim, S., Choi, Y., Matsunaga, D. E., and Kim, K.-E. Stitching sub-trajectories with conditional diffusion model for goal-conditioned offline rl. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 13160–13167, 2024.
- Kong, Y., Mao, H., Zhao, Q., Zhang, B., Ruan, J., Shen, L., Chang, Y., Wang, X., Zhao, R., and Tao, D. Qpo: Query-dependent prompt optimization via multi-loop offline reinforcement learning. *arXiv preprint arXiv:2408.10504*, 2024.
- Kong, Y., Ma, G., Zhao, Q., Wang, H., Shen, L., Wang, X., and Tao, D. Mastering massive multi-task reinforcement learning via mixture-of-expert decision transformer. *arXiv preprint arXiv:2505.24378*, 2025.
- Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit q-learning. *arXiv preprint arXiv:2110.06169*, 2021.
- Kumar, A., Zhou, A., Tucker, G., and Levine, S. Conservative q-learning for offline reinforcement learning. In *Advance in Neural Information Processing Systems*, 2020.
- Lee, J., Yun, S., Yun, T., and Park, J. Gta: Generative trajectory augmentation with guidance for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 37:56766–56801, 2024.
- Lee, K. and Choi, J. State-covering trajectory stitching for diffusion planners. *Advances in Neural Information Processing Systems*, 38:57273–57303, 2026.
- Levine, S., Kumar, A., Tucker, G., and Fu, J. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- Li, G., Shan, Y., Zhu, Z., Long, T., and Zhang, W. Diff-stitch: Boosting offline reinforcement learning with diffusion-based trajectory stitching. *arXiv preprint arXiv:2402.02439*, 2024.
- Li, S. and Zhang, X. Augmenting offline reinforcement learning with state-only interactions. *arXiv preprint arXiv:2402.00807*, 2024.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. Isolation forest. In *IEEE International Conference on Data Mining*, 2008.
- Liu, S., Qing, Y., Xu, S., Wu, H., Zhang, J., Cong, J., Chen, T., Liu, Y., and Song, M. Curricular subgoals for inverse reinforcement learning. *arXiv preprint arXiv:2306.08232*, 2023.
- Liu, S., Luo, W., Zhou, Y., Chen, K., Zhang, Q., Xu, H., Guo, Q., and Song, M. Transmission interface power flow adjustment: A deep reinforcement learning approach based on multi-task attribution map. *IEEE Transactions on Power Systems*, 39(2):3324–3335, 2024a.
- Liu, S., Song, J., Zhou, Y., Yu, N., Chen, K., Feng, Z., and Song, M. Interaction pattern disentangling for multi-agent reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):8157–8172, 2024b.
- Liu, S., Song, J., Zhou, Y., Yu, N., Chen, K., Feng, Z., and Song, M. Interaction pattern disentangling for multi-agent reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):8157–8172, 2024c.
- Lu, C., Ball, P., Teh, Y. W., and Parker-Holder, J. Synthetic experience replay. In *Advance in Neural Information Processing Systems*, 2023.
- Luo, Y., Mishra, U., Du, Y., and Xu, D. Generative trajectory stitching through diffusion composition. *Advances in Neural Information Processing Systems*, 38:37809–37843, 2026.

- Lyu, J., Ma, X., Li, X., and Lu, Z. Mildly conservative q-learning for offline reinforcement learning. In *Advance in Neural Information Processing Systems*, 2022.
- Nair, A., Gupta, A., Dalal, M., and Levine, S. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- Nishimori, S. Jax-corl: Clean single-file implementations of offline rl algorithms in jax. 2024. URL <https://github.com/nissymori/JAX-CORL>.
- Park, S., Frans, K., Levine, S., and Kumar, A. Is value learning really the main bottleneck in offline rl? *arXiv preprint arXiv:2406.09329*, 2024.
- Park, S., Frans, K., Eysenbach, B., and Levine, S. Ogbench: Benchmarking offline goal-conditioned rl. In *International Conference on Learning Representations*, volume 2025, pp. 94937–94982, 2025a.
- Park, S., Li, Q., and Levine, S. Flow q-learning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 48104–48127, 2025b.
- Paster, K., McIlraith, S., and Ba, J. You can’t count on luck: Why decision transformers and rvs fail in stochastic environments. In *Advance in Neural Information Processing Systems*, 2022.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers. *arXiv preprint arXiv:2212.09748*, 2022.
- Pimentel, M. A., Clifton, D. A., Clifton, L., and Tarassenko, L. A review of novelty detection. *Signal processing*, 99: 215–249, 2014.
- Qing, Y., Liu, S., Song, J., Wang, H., and Song, M. A survey on explainable reinforcement learning: Concepts, algorithms, challenges. *arXiv preprint arXiv:2211.06665*, 2022.
- Qing, Y., Liu, S., Cong, J., Chen, K., Zhou, Y., and Song, M. A2po: Towards effective offline reinforcement learning from an advantage-aware perspective. In *Advance in Neural Information Processing Systems*, 2024.
- Schmied, T., Paischer, F., Patil, V., Hofmarcher, M., Pascanu, R., and Hochreiter, S. Retrieval-augmented decision transformer: External memory for in-context rl. *arXiv preprint arXiv:2410.07071*, 2024.
- Sohn, K., Lee, H., and Yan, X. Learning structured output representation using deep conditional generative models. In *Advance in Neural Information Processing Systems*, 2015.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- Song, Y., Durkan, C., Murray, I., and Ermon, S. Maximum likelihood training of score-based diffusion models. In *Advances in Neural Information Processing Systems*, 2021.
- Su, Y., Kong, Y., and Wang, X. Uncertainty-aware data augmentation for offline reinforcement learning. In *2023 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. IEEE, 2023.
- Sutton, R. S., Barto, A. G., et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Wang, R., Frans, K., Abbeel, P., Levine, S., and Efros, A. A. Prioritized generative replay. *arXiv preprint arXiv:2410.18082*, 2024.
- Wang, Z., Hunt, J. J., and Zhou, M. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.
- Xia, B., Kong, Y., Chang, Y., Yuan, B., Li, Z., Wang, X., and Liang, B. Deer: A delay-resilient framework for reinforcement learning with variable delays. *arXiv preprint arXiv:2406.03102*, 2024.
- Xu, F., Liu, S., Qing, Y., Zhou, Y., Wang, Y., and Song, M. Temporal prototype-aware learning for active voltage control on power distribution networks. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024.
- Xu, H., Jiang, L., Li, J., Yang, Z., Wang, Z., Chan, V. W. K., and Zhan, X. Offline rl with no ood actions: In-sample learning via implicit value regularization. *arXiv preprint arXiv:2303.15810*, 2023.
- Yang, Q. and Wang, Y.-X. RTDiff: Reverse trajectory synthesis via diffusion for offline reinforcement learning. In *International Conference on Learning Representations*, 2025.
- Yu, H., Zhang, D., Du, Q., Zhao, Y., Zhang, H., Chen, G., Veas, E. E., and Zhao, J. Astro: Adaptive stitching via dynamics-guided trajectory rollouts. *arXiv preprint arXiv:2511.23442*, 2025.
- Yu, T., Thomas, G., Yu, L., Ermon, S., Zou, J. Y., Levine, S., Finn, C., and Ma, T. Mopo: Model-based offline policy optimization. In *Advance in Neural Information Processing Systems*, 2020.

- Yu, T., Kumar, A., Rafailov, R., Rajeswaran, A., Levine, S., and Finn, C. Combo: Conservative offline model-based policy optimization. In *Advance in Neural Information Processing Systems*, 2021.
- Yue, Y., Kang, B., Ma, X., Xu, Z., Huang, G., and Yan, S. Boosting offline reinforcement learning via data rebalancing. *arXiv preprint arXiv:2210.09241*, 2022.
- Zhang, J., Lyu, J., Ma, X., Yan, J., Yang, J., Wan, L., and Li, X. Uncertainty-driven trajectory truncation for data augmentation in offline reinforcement learning. *arXiv preprint arXiv:2304.04660*, 2023.
- Zhao, Z., Ren, Z., Yang, L., Liang, Y., Yuan, F., Ren, P., Chen, Z., Ma, J., and Xin, X. Offline trajectory optimization for offline reinforcement learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 4002–4013, 2025.
- Zhou, Y., Zheng, Y., Hu, Y., Chen, K., Zheng, T., Song, J., Song, M., and Liu, S. Cooperative policy agreement: Learning diverse policy for offline marl. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 23018–23026, 2025.
- Zhou, Z., Zhu, C., Zhou, R., Cui, Q., Gupta, A., and Du, S. Free from bellman completeness: Trajectory stitching via model-based return-conditioned supervised learning. In *International Conference on Learning Representations*, volume 2024, pp. 3687–3711, 2024.

A. Pseudocode

The pseudocode of our BiTrajDiff method is provided in Algorithm 1.

Algorithm 1 Bidirectional Trajectory Diffusion (BiTrajDiff) data augmentation pipeline.

Input: dataset $\mathcal{D}$, ratio σ , horizon H , target return-to-go R^f and R^b , filter number C_{ood} and C_{greedy} .

Initialize: Forward and backward diffusion denoise network ϵ_{θ^f} and ϵ_{θ^b} , inverse dynamics model f_ψ , reward network r_ϕ , and the augmented dataset $\mathcal{D}_{\text{da}} = \emptyset$

▷ BiTrajDiff Model Training

Train the forward and backward trajectory diffusion model ϵ_{θ^f} and ϵ_{θ^b} with $\mathcal{D}$ according to Eq. 4.

Train the inverse dynamics model f_ψ and reward model r_ϕ with $\mathcal{D}$ according to Eq. 6

▷ BiTrajDiff Data Generation

While $|\mathcal{D}_{\text{da}}| \leq \sigma * |\mathcal{D}|$ **do**

 # Bidirectional State Trajectory Generation

 Sample random batch of states $\mathcal{B} = \{s_t\} \sim \mathcal{D}$.

 Generate $\mathbf{x}_0^f(\tau) = \{\hat{s}_t, \hat{s}_{t+1}, \dots, \hat{s}_{t+H-1}\}$ with ϵ_{θ^f} conditioned on s_t and R^f .

 Generate $\mathbf{x}_0^b(\tau) = \{\hat{s}_{t-H+1}, \dots, \hat{s}_{t-1}, \hat{s}_t\}$ with ϵ_{θ^b} conditioned on s_t and R^b .

 Construct the bidirectional state trajectory $\hat{\tau}_s(s_t)$ with $\mathbf{x}_0^b(\tau)$ and $\mathbf{x}_0^f(\tau)$ according to Eq. 5.

 # Bidirectional Trajectory Completion

 Compute the action $\hat{a}_i$ and reward $\hat{r}_i$ signal with $\hat{\tau}_s(s_t)$ by f_ψ and r_ϕ .

 Obtain the generated trajectory set $\hat{\mathcal{D}}$ containing $\hat{\tau}(s_t) = \{(\hat{s}_i, \hat{a}_i, \hat{r}_i, \hat{s}_{i+1})\}_{i=t-H+1}^{t+H-2}$

 # Bidirectional Trajectory Filter

 Filter the Top- C_{ood} trajectories with smallest $d(\hat{\tau})$ from $\hat{\mathcal{D}}$

 Filter the Top- C_{greedy} trajectories as batch $\tilde{\mathcal{D}}$ with largest $R(\hat{\tau})$ from the Top- C_{ood} trajectories.

 Add the final generated batch into the augmented dataset: $\mathcal{D}_{\text{da}} = \mathcal{D}_{\text{da}} \cup \tilde{\mathcal{D}}$

End while

B. Experiments Details

B.1. Task Abbreviations and Task Versions

For improved readability and conciseness, we use abbreviations for the locomotion tasks throughout the main text, with the corresponding definitions provided in Table S1.

B.2. Implementation Details

B.2.1. BITRAJDIF DIFF IMPLEMENTATION

In this section, we provide the implementation details of our experiments. We conducted our experiments on a cluster of 4 A100 GPUs. The source code will be made publicly available upon the publication of this paper. Our BiTrajDiff is implemented based on CleanDiffuser (Dong et al., 2024), a popular modularized library for diffusion models in decision making. We represent the noise model ϵ_θ with the 1D DiT backbone (Peebles & Xie, 2022), consisting 2 transformer blocks with adaptive layer normalization. Each block contains a multi-head self-attention layer followed by a feed-forward MLP. The diffusion timestep embeddings are first encoded via Fourier features and then projected through a two-layer MLP network with 128 hidden units. Similarly, the condition inputs are encoded by a two-layer MLP network with 128

Table S1. Abbreviations of the corresponding locomotion tasks and datasets.

Dataset	halfcheetah	walker2d	hopper
medium	hc-m	w-m	h-m
medium-replay	hc-m-r	w-m-r	h-m-r
medium-expert	hc-m-e	w-m-e	h-m-e

BiTrajDiff: Bidirectional Trajectory Generation with Diffusion Models for Offline Reinforcement Learning

Table S2. Test returns of our proposed BiTrajDiff and baselines on the Gym tasks. $\pm$ corresponds to the standard deviation of the performance on 5 random seeds. The best and the second-best results of each setting are marked as **bold** and underline, respectively.

Source	Task	CQL (Kumar et al., 2020)					DT (Chen et al., 2021)				
		Base	RTDiff	Synther	DiffStitch	Ours	Base	RTDiff	Synther	DiffStitch	Ours
medium	halfcheetah	47.4 $\pm$ 0.2	47.8 $\pm$ 1.6	48.3 $\pm$ 0.4	48.0 $\pm$ 0.2	48.7$\pm$0.3	42.7 $\pm$ 0.5	42.6 $\pm$ 2.5	42.7 $\pm$ 0.1	42.8 $\pm$ 0.4	43.3$\pm$0.2
	walker2d	82.6 $\pm$ 1.0	<u>83.5$\pm$1.7</u>	83.4 $\pm$ 0.5	82.4 $\pm$ 4.5	84.2$\pm$0.5	69.6 $\pm$ 11.0	71.6 $\pm$ 10.2	74.8 $\pm$ 4.3	72.3 $\pm$ 9.4	77.8$\pm$4.0
	hopper	68.8 $\pm$ 4.1	70.9 $\pm$ 3.7	<u>75.5$\pm$8.4</u>	69.7 $\pm$ 5.8	80.9$\pm$3.4	56.0 $\pm$ 1.9	56.6 $\pm$ 5.1	<u>58.1$\pm$2.4</u>	58.4$\pm$2.4	57.8 $\pm$ 2.2
medium expert	halfcheetah	87.2 $\pm$ 4.6	90.0 $\pm$ 2.9	<u>92.2$\pm$3.5</u>	91.4 $\pm$ 5.0	94.1$\pm$1.1	70.9 $\pm$ 12.3	72.3 $\pm$ 7.0	80.4 $\pm$ 9.8	76.9 $\pm$ 12.7	83.5$\pm$6.3
	walker2d	<u>110.2$\pm$0.6</u>	110.1 $\pm$ 0.8	110.1 $\pm$ 0.4	109.4 $\pm$ 3.6	110.4$\pm$0.4	104.6 $\pm$ 6.9	101.2 $\pm$ 1.8	<u>105.7$\pm$3.5</u>	104.0 $\pm$ 8.6	106.3$\pm$6.0
	hopper	103.5 $\pm$ 8.1	102.2 $\pm$ 3.2	103.2 $\pm$ 10.2	<u>105.3$\pm$7.8</u>	105.7$\pm$5.8	107.5 $\pm$ 3.4	<u>109.6$\pm$1.3</u>	109.2 $\pm$ 0.5	108.8 $\pm$ 4.8	110.1$\pm$1.9
medium replay	halfcheetah	45.9 $\pm$ 0.2	45.6 $\pm$ 0.4	47.4$\pm$0.6	45.1 $\pm$ 0.4	45.8 $\pm$ 0.3	38.8 $\pm$ 2.4	39.1 $\pm$ 4.1	39.3 $\pm$ 1.3	38.9 $\pm$ 2.6	39.5$\pm$0.8
	walker2d	82.1 $\pm$ 1.8	<u>84.4$\pm$3.2</u>	84.8$\pm$0.6	82.4 $\pm$ 1.8	84.8$\pm$1.5	52.1 $\pm$ 5.5	53.8 $\pm$ 2.4	<u>56.1$\pm$5.2</u>	54.6 $\pm$ 7.5	58.2$\pm$6.1
	hopper	95.7 $\pm$ 2.5	95.9 $\pm$ 7.1	96.1 $\pm$ 1.2	<u>98.2$\pm$6.8</u>	100.3$\pm$1.3	67.3 $\pm$ 8.9	70.2$\pm$10.6	69.0 $\pm$ 16.2	68.1 $\pm$ 19.7	<u>69.6$\pm$13.1</u>
Average		80.4	81.2	82.3	81.3	83.9	67.7	68.6	70.6	69.4	71.8

Table S3. Test returns of our proposed BiTrajDiff and baselines on Maze and Franka Kitchen tasks.

Task	CQL (Kumar et al., 2020)					DT (Chen et al., 2021)				
	Base	RTDiff	Synther	DiffStitch	Ours	Base	RTDiff	Synther	DiffStitch	Ours
maze2d-umaze	4.5 $\pm$ 1.1	<u>11.3$\pm$8.3</u>	2.5 $\pm$ 4.6	2.3 $\pm$ 1.4	18.6$\pm$9.4	24.9 $\pm$ 10.6	<u>29.8$\pm$14.5</u>	31.4 $\pm$ 9.5	22.9 $\pm$ 2.7	36.1$\pm$8.7
maze2d-medium	84.4 $\pm$ 24.5	87.7 $\pm$ 23.2	<u>90.5$\pm$10.7</u>	72.9 $\pm$ 16.8	95.8$\pm$17.4	16.6 $\pm$ 2.2	18.8 $\pm$ 2.7	<u>24.7$\pm$3.1</u>	19.3 $\pm$ 1.6	26.6$\pm$4.0
maze2d-large	34.3 $\pm$ 25.5	38.9 $\pm$ 20.7	45.3 $\pm$ 14.2	65.2$\pm$72.5	<u>48.3$\pm$28.7</u>	22.3 $\pm$ 12.5	<u>24.9$\pm$4.9</u>	24.2 $\pm$ 6.7	24.4 $\pm$ 7.3	28.5$\pm$4.4
antmaze-umaze-diverse	31.6 $\pm$ 8.4	40.6 $\pm$ 9.0	<u>42.4$\pm$6.8</u>	39.2 $\pm$ 9.3	48.8$\pm$4.4	42.0 $\pm$ 4.9	<u>46.4$\pm$16.7</u>	44.8 $\pm$ 11.2	42.6 $\pm$ 4.7	47.8$\pm$2.9
antmaze-medium-diverse	57.2 $\pm$ 16.2	60.4 $\pm$ 12.8	62.6 $\pm$ 9.8	<u>63.6$\pm$12.6</u>	73.2$\pm$11.6	0.0 $\pm$ 0.0	<u>0.2$\pm$0.4</u>	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.4$\pm$0.5
antmaze-large-diverse	8.0 $\pm$ 7.3	5.3 $\pm$ 1.2	<u>12.6$\pm$10.9</u>	2.6 $\pm$ 3.7	16.0$\pm$10.2	<u>0.0$\pm$0.0</u>	<u>0.0$\pm$0.0</u>	0.0 $\pm$ 0.0	0.8$\pm$0.7	0.0 $\pm$ 0.0
Maze Mean	36.7	40.7	42.7	41.0	50.1	17.6	20.0	20.8	18.3	23.2
kitchen-complete	7.8 $\pm$ 11.9	15.2$\pm$9.3	8.4 $\pm$ 6.1	10.5 $\pm$ 5.3	<u>13.8$\pm$7.4</u>	61.7 $\pm$ 14.6	61.3 $\pm$ 8.0	62.3 $\pm$ 8.6	<u>63.4$\pm$11.7</u>	67.9$\pm$6.8
kitchen-partial	21.1 $\pm$ 2.5	25.8$\pm$3.1	23.2 $\pm$ 7.3	19.3 $\pm$ 6.7	<u>24.9$\pm$4.7</u>	21.6 $\pm$ 11.1	22.2 $\pm$ 7.5	21.8 $\pm$ 7.5	29.2 $\pm$ 5.8	33.5$\pm$18.9
kitchen-mixed	16.2 $\pm$ 6.4	19.1 $\pm$ 1.2	<u>19.3$\pm$5.7</u>	12.0 $\pm$ 9.6	21.5$\pm$7.9	22.5 $\pm$ 16.5	23.7 $\pm$ 14.7	31.6 $\pm$ 14.8	24.7 $\pm$ 10.4	37.0$\pm$11.7
Kitchen Mean	15.0	20.0	17.0	13.9	20.1	35.3	35.7	38.6	39.1	46.1

hidden units. For the sampling process, we follow the CleanDiffuser reimplementation of Decision Diffuser (Ajay et al., 2022), employing the VP-SDE (Song et al., 2020) formulation with 20 diffusion steps for sampling. Meanwhile, the inverse dynamics and rewards model are both represented by a two-layer MLP network with 512 hidden units, respectively. During BiTrajDiff training, we employ an Adam optimizer with a learning rate of $2e-4$ for all networks, using a batch size of 64 and performing $1e6$ gradient steps in total. The default trajectory horizon H for both training and generation is set to 5. The ratio σ of the augmented dataset is 30%. During our BiTrajDiff data augmentation pipeline, the batch size is set to 512, while the filter number C_{ood} is 256 and C_{greedy} is 64. As for the target return-to-go R^f and R^b , we set them to be equal in our BiTrajDiff conditional generation. Each task is assigned its own target return, and in our experiments we directly adopt the corresponding hyperparameters from the CleanDiffuser reimplementation of Decision Diffuser (Ajay et al., 2022).

B.2.2. OFFLINE RL ALGORITHM IMPLEMENTATION

In our experiments, we evaluate BiTrajDiff against four offline RL algorithms: IQL, TD3BC, CQL, and DT. To enable efficient and large-scale evaluation, we conduct all offline RL experiments using the JAX-based library JAX-CORL (Nishimori, 2024). **For fairness, we report re-run results on JAX-CORL with initial datasets as baseline performance, rather than the scores reported in the original papers, to account for discrepancies across codebases.** Meanwhile, for each offline RL algorithm, we ensure that the hyperparameters are configured consistently with those reported in the respective original papers.

C. Results on CQL and DT

Results for Locomotion Tasks. The results on D4RL locomotion tasks are shown in Table S2. BiTrajDiff achieves the highest average returns in all of the environments under both CQL and DT baselines, with improvements of 4.4% and 6.1%, respectively. At the same time, BiTrajDiff maintains high stability across all locomotion tasks, substantially exceeding other tested DA methods. These results highlight the effectiveness of BiTrajDiff.

Results for Maze and Franka Kitchen Tasks. Table S3 shows the experimental results on the D4RL Maze and Franka Kitchen tasks. BiTrajDiff acquires the highest average return across all tested tasks of sparse reward scenarios under both CQL and DT baselines. Meanwhile, BiTrajDiff maintains higher stability in performance improvements across all the settings than other DA methods like RTDiff. This observation confirms that BiTrajDiff can synthesize novel and high-quality long-horizon trajectories by stitching forward-future and backward-history paths to address the challenge of the availability of high-quality data under sparse-reward settings. This result underlines the efficacy and applicability of BiTrajDiff in enhancing data diversity and quality.

D. Extended Discussion on Trajectory Stitching Methods

Related Work Discussion. Recent trajectory stitching methods for offline reinforcement learning (Liu et al., 2024c; Zhou et al., 2025; Liu et al., 2024b; Xia et al., 2024) can be broadly categorized into two main paradigms. The first paradigm is *model-based stitching*, where learned dynamics models are used to generate trajectories step by step in order to connect separated states or sub-trajectories. Representative methods include MBRC SL (Zhou et al., 2024) and ASTRO (Yu et al., 2025). This paradigm utilizes a learned dynamic model to naturally connect distant states by rollouts and thereby recover long-range trajectory-level connectivity absent from the logged dataset. However, this flexibility also introduces error propagation due to auto-regressive rollout. Consequently, although model-based stitching can expand the apparent support of the dataset, the generated bridge may suffer from degraded local dynamic validity, especially near precise stitching points or in regions where the learned model is extrapolating beyond well-supported transitions. The second paradigm is *direct middle-segment generation*, where the method either directly synthesizes intermediate trajectory segments between endpoints. DiffStitch (Li et al., 2024), SSD (Kim et al., 2024), SCoTS (Lee & Choi, 2026), and CD/CompDiffuser (Luo et al., 2026) are representative examples of this direction. By explicitly modeling missing middle segments, these methods can improve goal-reaching behavior, trajectory diversity, and compositional generalization. Nevertheless, strict dynamic consistency at the exact stitching points remains an inherent challenge. The generated segment must be simultaneously compatible with the incoming endpoint, the outgoing endpoint, and the environment dynamics; when the offline data provide only sparse evidence for such a connection, satisfying all three constraints becomes difficult even if the global trajectory appears semantically plausible.

Mechanistic Advantage of BiTrajDiff. Instead of relying on single-direction autoregressive rollouts or unconstrained middle-segment generation, BiTrajDiff uses an anchor-centered factorization to recover missing trajectory-level couplings. Given an anchor state, it independently generates a backward-history segment and a forward-future segment, and stitches them at the shared anchor s_t . This factorization is an operational design principle rather than an assumption of true conditional independence. It enables BiTrajDiff to compose locally supported incoming and outgoing behavior patterns whose pairing may be absent in the dataset, while avoiding long autoregressive rollouts that accumulate prediction errors. The completed trajectories are further screened by the OOD and greedy filters, which respectively discourage unsupported transitions and retain high-return candidates. Thus, BiTrajDiff expands trajectory-level support while preserving local dynamic consistency.

Empirical Validation. The empirical comparisons further support this mechanistic interpretation. Table S4 evaluates the stitching and compositional connectivity recovery ability of BiTrajDiff on selected OGBench (Park et al., 2025a) single-task offline RL problems, where the compared baselines include the single-direction method SynthER (Lu et al., 2023) and representative stitching methods DiffStitch (Li et al., 2024), ASTRO (Yu et al., 2025), and SSD (Kim et al., 2024). Under both IQL and FQL backbones, BiTrajDiff achieves the strongest average performance, indicating that the shared-anchor bidirectional generation mechanism more effectively recovers missing trajectory-level connectivity than methods that rely on single-direction generation, autoregressive rollout, or direct segment stitching alone. Table S5 further extends this comparison to goal-conditioned offline RL, where BiTrajDiff is evaluated against SynthER, SCoTS (Lee & Choi, 2026), and CD/CompDiffuser (Luo et al., 2026). Its leading average performance under both GCIQL and CRL shows that the same mechanism remains effective when the policy must compose trajectories toward explicit goals in stitching and exploration.

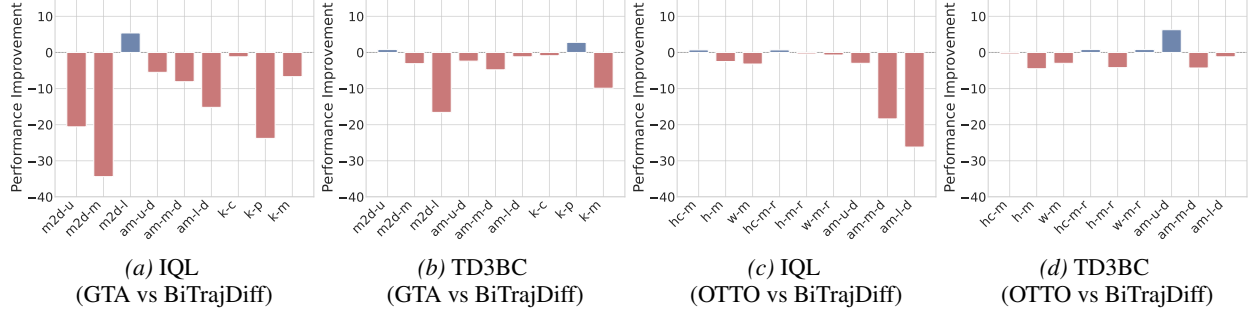


Figure S1. Performance enhancement comparison with GTA (Lee et al., 2024) and OTTO (Zhao et al., 2025) on IQL and TD3BC.

Table S4. Offline RL performance comparison under our proposed BiTrajDiff and baselines on selected single-task OGBench tasks.

Task	IQL (Kostrikov et al., 2021)						FQL (Park et al., 2025b)					
	Base	ASTRO	DiffStitch	Synther	SSD	Ours	Base	ASTRO	DiffStitch	Synther	SSD	Ours
antmaze-large-stitch-singletask-task1	26.2	51.7	35.0	31.1	<u>53</u> $\pm$ 7	64 $\pm$ 10	29.2	57.3	33.1	28.7	<u>75</u> $\pm$ 22	79 $\pm$ 13
humanoidmaze-medium-stitch-singletask-task1	29.7	31.4	28.3	31.2	<u>33</u> $\pm$ 11	38 $\pm$ 7	17.5	30.0	22.6	15.9	<u>43</u> $\pm$ 16	47 $\pm$ 18
cube-single-play-singletask-task1	81.5	89.2	79.0	82.4	<u>82</u> $\pm$ 7	87 $\pm$ 5	88.0	92.9	89.6	87.3	<u>93</u> $\pm$ 4	98 $\pm$ 4
Average	45.8	57.4	47.4	48.2	56.0	63.0	44.9	60.1	48.4	44.0	70.3	74.7

Table S5. Goal-conditioned offline RL performance comparison under our proposed BiTrajDiff and baselines on OGBench tasks.

Task	GCIQL (Kostrikov et al., 2021)					CRL (Kumar et al., 2020)				
	Base	Synther	SCoTS	CD	Ours	Base	Synther	SCoTS	CD	Ours
antmaze-medium-stitch-v0	29 $\pm$ 6	31 $\pm$ 3	<u>35</u> $\pm$ 2	34 $\pm$ 4	38 $\pm$ 2	53 $\pm$ 6	48 $\pm$ 3	<u>65</u> $\pm$ 3	61 $\pm$ 4	68 $\pm$ 9
antmaze-large-stitch-v0	<u>7</u> $\pm$ 2	3 $\pm$ 4	<u>7</u> $\pm$ 1	7 $\pm$ 3	8 $\pm$ 4	11 $\pm$ 2	12 $\pm$ 2	<u>19</u> $\pm$ 1	<u>17</u> $\pm$ 2	19 $\pm$ 2
antmaze-medium-explore-v0	13 $\pm$ 2	12 $\pm$ 3	<u>18</u> $\pm$ 3	17 $\pm$ 5	19 $\pm$ 4	3 $\pm$ 2	3 $\pm$ 1	15 $\pm$ 3	11 $\pm$ 2	<u>14</u> $\pm$ 2
Average	16.3	15.3	<u>20.0</u>	19.3	21.7	22.3	21.0	<u>33.0</u>	29.7	33.7

environments. Finally, Figure S1 compares BiTrajDiff with two additional DA baselines, the diffusion-based method GTA (Lee et al., 2024) and the world-model-based method OTTO (Zhao et al., 2025), under the IQL and TD3BC backbones. Although GTA and OTTO can improve performance in some settings, their gains are generally smaller and less consistent, whereas BiTrajDiff achieves more robust improvements by synthesizing useful long-horizon transitions while controlling trajectory inconsistency through completion and filtering. Taken together, Table S4, Table S5, and Figure S1 show that effective trajectory stitching requires both support expansion and dynamic consistency: the augmented data should introduce new connections between behavior fragments, which remain locally compatible with the environment dynamics.

E. Ablation Study on the Direction of Diffusion Generation

We conduct experiments with different augmented diffusion trajectories generated in single-directional and bi-directional. Table S6 shows the test returns of the IQL and TD3BC algorithms. For CQL and DT algorithms, the quantitative results are reported in Table S7. The algorithms were evaluated on D4RL locomotion tasks with augmented trajectories generated in single-direction and bi-directional. Compared to the base method, the forward method offers only modest gains and is at times even detrimental to performance. Conversely, the backward method generally outperforms the base method across most tasks, achieving substantial gains. Finally, our bi-directional approach, BiTrajDiff, consistently exhibits superior and robust performance. Our BiTrajDiff enables global connectivity between originally unreachable states from the original dataset and thus constructs novel and high-quality augmented trajectories.

Table S6. Test returns of our proposed BiTrajDiff with augmented single- and bi-directional diffusion generated trajectories on IQL and TD3BC algorithms under D4RL locomotion tasks.

Source	Task	IQL (Kostrikov et al., 2021)				TD3BC (Fujimoto & Gu, 2021)			
		Base	Forward	Backward	Ours	Base	Forward	Backward	Ours
medium	halfcheetah	48.2 $\pm$ 0.2	47.3 $\pm$ 0.8	48.4 $\pm$ 0.5	48.6$\pm$0.2	48.4 $\pm$ 0.2	50.0 $\pm$ 0.3	50.8$\pm$0.7	<u>50.3$\pm$0.4</u>
	walker2d	77.6 $\pm$ 5.3	79.1 $\pm$ 4.9	<u>83.7$\pm$2.6</u>	86.7$\pm$1.7	82.0 $\pm$ 2.7	81.7 $\pm$ 0.7	84.6 $\pm$ 0.4	86.7$\pm$1.0
	hopper	67.0 $\pm$ 1.9	70.2 $\pm$ 7.6	<u>75.8$\pm$4.8</u>	81.1$\pm$5.0	60.4 $\pm$ 3.7	65.2 $\pm$ 6.7	80.3$\pm$4.8	<u>79.0$\pm$3.5</u>
medium expert	halfcheetah	90.1 $\pm$ 4.4	92.5 $\pm$ 1.2	94.2 $\pm$ 0.1	95.3$\pm$0.1	92.3 $\pm$ 5.1	94.7 $\pm$ 1.9	<u>95.3$\pm$0.9</u>	96.3$\pm$0.4
	walker2d	<u>112.4$\pm$0.4</u>	109.7 $\pm$ 1.6	112.5$\pm$0.4	112.5$\pm$0.5	109.3 $\pm$ 0.2	109.1 $\pm$ 0.5	<u>110.0$\pm$0.4</u>	110.2$\pm$0.3
	hopper	105.4 $\pm$ 6.3	102.3 $\pm$ 7.0	<u>109.1$\pm$5.2</u>	110.9$\pm$0.6	94.2 $\pm$ 11.1	88.7 $\pm$ 5.5	<u>108.4$\pm$2.5</u>	109.0$\pm$5.1
medium replay	halfcheetah	43.5 $\pm$ 0.2	42.5 $\pm$ 0.8	<u>43.7$\pm$0.1</u>	44.1$\pm$0.2	44.1 $\pm$ 0.1	44.4 $\pm$ 0.3	45.1$\pm$0.5	<u>45.0$\pm$0.6</u>
	walker2d	73.3 $\pm$ 7.5	79.4 $\pm$ 5.0	<u>82.2$\pm$2.9</u>	87.6$\pm$2.2	80.9 $\pm$ 6.0	87.2 $\pm$ 2.4	85.8 $\pm$ 4.9	89.9$\pm$1.4
	hopper	94.7 $\pm$ 4.0	<u>97.2$\pm$3.0</u>	89.6 $\pm$ 3.1	102.8$\pm$0.6	64.4 $\pm$ 8.9	83.5 $\pm$ 12.9	<u>83.8$\pm$9.7</u>	85.0$\pm$10.3
Average		79.1	80.0	82.1	85.5	75.1	78.3	82.7	83.5

Table S7. Test returns of our proposed BiTrajDiff with augmented single- and bi-directional diffusion generated trajectories on CQL and DT algorithms under D4RL locomotion tasks.

Source	Task	CQL (Kumar et al., 2020)				DT (Chen et al., 2021)			
		Base	Forward	Backward	Ours	Base	Forward	Backward	Ours
medium	halfcheetah	47.4 $\pm$ 0.2	47.5 $\pm$ 0.2	48.9$\pm$0.4	<u>48.7$\pm$0.3</u>	42.7 $\pm$ 0.5	42.7 $\pm$ 0.2	43.0 $\pm$ 0.3	43.3$\pm$0.2
	walker2d	82.6 $\pm$ 1.0	80.9 $\pm$ 2.7	83.8 $\pm$ 0.7	84.2$\pm$0.5	69.6 $\pm$ 11.0	74.6 $\pm$ 3.6	<u>76.1$\pm$3.5</u>	77.8$\pm$4.0
	hopper	68.8 $\pm$ 4.1	71.6 $\pm$ 14.9	<u>75.2$\pm$9.5</u>	80.9$\pm$3.4	56.0 $\pm$ 1.9	56.6 $\pm$ 5.6	<u>56.9$\pm$5.7</u>	57.8$\pm$2.2
medium expert	halfcheetah	87.2 $\pm$ 4.6	88.0 $\pm$ 3.9	<u>92.6$\pm$2.2</u>	94.1$\pm$1.1	70.8 $\pm$ 12.3	<u>74.3$\pm$4.7</u>	64.7 $\pm$ 4.4	83.5$\pm$6.3
	walker2d	110.2 $\pm$ 0.6	110.5$\pm$0.6	109.5 $\pm$ 0.7	<u>110.4$\pm$0.4</u>	104.6 $\pm$ 6.9	<u>105.1$\pm$5.1</u>	104.9 $\pm$ 4.8	106.3$\pm$6.0
	hopper	<u>103.5$\pm$8.1</u>	102.7 $\pm$ 13.9	102.5 $\pm$ 7.8	105.7$\pm$5.8	107.5 $\pm$ 3.4	108.3 $\pm$ 3.3	<u>109.8$\pm$2.4</u>	110.1$\pm$1.9
medium replay	halfcheetah	45.9 $\pm$ 0.2	45.9 $\pm$ 0.5	46.2$\pm$0.4	45.8 $\pm$ 0.3	38.8 $\pm$ 2.4	36.4 $\pm$ 2.9	39.3 $\pm$ 0.5	39.5$\pm$0.8
	walker2d	82.1 $\pm$ 1.8	<u>82.9$\pm$0.6</u>	84.4 $\pm$ 2.1	84.8$\pm$1.5	52.1 $\pm$ 5.5	58.6$\pm$10.0	54.9 $\pm$ 4.2	<u>58.2$\pm$6.1</u>
	hopper	95.7 $\pm$ 2.5	<u>97.9$\pm$6.1</u>	93.8 $\pm$ 5.3	100.3$\pm$1.3	67.3 $\pm$ 8.9	62.1 $\pm$ 13.0	<u>68.4$\pm$17.2</u>	69.6$\pm$13.1
Average		80.4	80.9	81.9	83.9	67.7	68.7	68.7	71.8

Table S8. Test results of BiTrajDiff with data from different variants related to trajectory filters. OOD indicates the OOD filter, and G represents the greedy filter.

Source	Task	IQL (Kostrikov et al., 2021)					TD3BC (Fujimoto & Gu, 2021)				
		Base	None	w G, w/o OOD	w OOD, w/o G	Ours	Base	None	w G, w/o OOD	w OOD, w/o G	Ours
medium	walker2d	73.3 $\pm$ 7.5	78.5 $\pm$ 10.2	83.9 $\pm$ 4.5	80.1 $\pm$ 7.0	87.6$\pm$2.2	80.9 $\pm$ 6.0	77.2 $\pm$ 24.4	77.5 $\pm$ 26.9	82.5 $\pm$ 8.0	89.9$\pm$1.4
	replay hopper	94.7 $\pm$ 4.0	93.6 $\pm$ 12.0	91.6 $\pm$ 8.2	<u>96.3$\pm$3.5</u>	102.8$\pm$0.6	64.4 $\pm$ 8.9	69.0 $\pm$ 19.2	<u>75.0$\pm$17.0</u>	74.6 $\pm$ 23.6	85.0$\pm$10.3

 Table S9. Dynamic error $\mathcal{E}_{\text{Dyn}}$ and L2 distance $\mathcal{E}_{\text{L2D}}$ comparison of trajectories generated by BiTrajDiff with different filtering strategies.

$\mathcal{E}_{\text{Dyn}} \downarrow / \mathcal{E}_{\text{L2D}} \uparrow$	w/o OOD + Greedy	w OOD, w/o Greedy	w Greedy, w/o OOD	w OOD + Greedy Filter (Ours)
halfcheetah-m-e	0.29 / 9.39	0.15 / 8.73	0.36 / 11.27	0.18 / 10.45
hopper-m-e	0.44 / 15.04	0.23 / 15.61	0.71 / 18.58	0.30 / 16.89
walker2d-m-e	0.34 / 11.26	0.22 / 10.36	0.47 / 14.97	0.26 / 14.10

Table S10. Performance comparison of BiTrajDiff with different augmented data ratios σ on the *walker2d-medium-replay*. The best result for each offline RL algorithm is marked as **bold**.

Ratio	IQL	TD3BC	CQL
0%	73.3 $\pm$ 7.5	80.9 $\pm$ 6.0	82.1 $\pm$ 1.8
10%	78.7 $\pm$ 2.4	82.0 $\pm$ 5.5	84.0 $\pm$ 3.1
20%	84.2 $\pm$ 1.3	87.5 $\pm$ 1.0	83.9 $\pm$ 2.0
30%	87.6 $\pm$ 2.2	89.9 $\pm$ 1.4	84.8 $\pm$ 1.5
40%	88.7 $\pm$ 4.5	86.7 $\pm$ 2.9	84.7 $\pm$ 2.1
50%	81.6 $\pm$ 5.6	86.4 $\pm$ 2.7	86.3 $\pm$ 2.1
100%	84.5 $\pm$ 10.2	86.7 $\pm$ 2.4	83.9 $\pm$ 3.3

Table S11. Test returns of our proposed BiTrajDiff and baselines with varying length of n -step TD estimator (n) on *walker2d-medium-replay*.

Length of n-step TD Estimator(n)	IQL (Kostrikov et al., 2021)				TD3BC (Fujimoto & Gu, 2021)			
	base	RTDiff	diff stitch	ours	base	RTDiff	diff stitch	ours
$n = 2$	73.3 $\pm$ 7.5	74.0 $\pm$ 5.2	74.9 $\pm$ 8.2	87.6 $\pm$ 2.2	80.9 $\pm$ 6.0	83.5 $\pm$ 7.4	80.5 $\pm$ 5.2	89.9 $\pm$ 1.4
$n = 5$	71.7 $\pm$ 12.4	76.4 $\pm$ 16.6	79.8 $\pm$ 2.4	86.7 $\pm$ 0.9	80.3 $\pm$ 12.0	88.6 $\pm$ 3.6	87.7 $\pm$ 4.0	87.3 $\pm$ 1.3
$n = 10$	69.5 $\pm$ 13.5	73.1 $\pm$ 12.7	71.4 $\pm$ 9.9	79.5 $\pm$ 2.3	81.5 $\pm$ 7.6	79.6 $\pm$ 12.7	81.3 $\pm$ 5.1	87.5 $\pm$ 3.4
$n = 15$	72.7 $\pm$ 11.4	67.2 $\pm$ 14.8	69.7 $\pm$ 6.0	77.9 $\pm$ 6.2	75.1 $\pm$ 9.9	80.6 $\pm$ 13.8	79.1 $\pm$ 9.7	83.9 $\pm$ 4.7

F. Effectiveness of Trajectory Filters

We illustrate the necessity of integrating both OOD and greedy filters in our full BiTrajDiff model. Without any filters, the performance is comparable to, and in some cases inferior to, that of the base method on both the IQL and TD3BC algorithms. When applying the greedy filter in isolation, the performance improvement is noticeable but marred by volatility. In contrast, using the OOD filter alone yields a more stable and substantial performance enhancement. Nevertheless, our full BiTrajDiff model, which integrates both filters, consistently outperforms all other configurations across the tested settings, indicating the necessity of both OOD and greedy filters.

To further investigate the mechanism of our trajectory filters, we quantitatively measure the dynamic error $\mathcal{E}_{\text{Dyn}}$ and L2 distance $\mathcal{E}_{\text{L2D}}$ of trajectories generated by BiTrajDiff with different filter combinations, as shown in Table S9. The results show that removing both filters leads to larger dynamic errors, indicating that unfiltered generated trajectories can contain inconsistent transitions. Using only the OOD filter substantially reduces dynamic error, but also limits trajectory diversity, as reflected by smaller L2 distances. Using only the greedy filter increases L2 distance, but introduces much larger dynamic errors. In contrast, BiTrajDiff with both OOD and greedy filters achieves a better balance: it maintains comparable dynamic consistency while preserving larger trajectory-level novelty.

G. Ablation Study on Augmented Data Ratio σ

In this section, we conduct experiments to assess the impact of the augmented data ratio σ . As shown in Table S10, when the ratio σ is too low, which is between 0% $\sim$ 20%, performance improvements remain consistently constrained. Conversely, while an augmentation ratio σ between 30% and 50% already provides a substantial and stable performance improvement, increasing σ to 100% compromises this stability, indicating a decline in the performance. This result demonstrates that both the limited and overabundant synthetic data have detrimental effects on the performance of BiTrajDiff, since the limited data is not enough to provide offline RL algorithms with new behavior patterns, and overabundant data introduces excessive noise, which hinders offline RL algorithms from learning effectively, and an optimal trade-off is achieved with a 30% to 50% augmentation range. As a result, we choose $\sigma = 30\%$ as the fixed hyperparameter for all our experiments.

H. Performance under n -step TD Estimator

In this section, we demonstrate the effectiveness of BiTrajDiff in Offline RL algorithms with varying lengths of the n -step TD estimators. As shown in Table S11 and Table S12, while RTDiff and diffstitch initially outperform the base method,

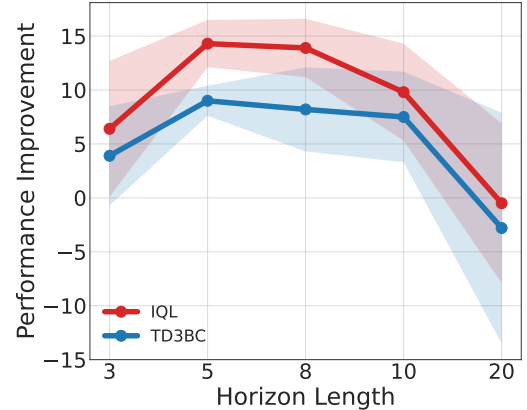
Table S12. Test returns of our proposed BiTrajDiff and baselines with varying length of n -step TD estimator (n) on *hopper-medium-replay*.

Length of n -step TD Estimator(n)	IQL (Kostrikov et al., 2021)				TD3BC (Fujimoto & Gu, 2021)			
	base	RTDiff	diff stitch	ours	base	RTDiff	diff stitch	ours
$n = 2$	94.7 $\pm$ 4.0	97.7 $\pm$ 2.7	96.4 $\pm$ 5.6	102.8$\pm$0.6	64.4 $\pm$ 8.9	72.1 $\pm$ 18.9	71.5 $\pm$ 21.0	85.0$\pm$10.3
$n = 5$	90.5 $\pm$ 1.9	94.7 $\pm$ 5.4	99.9$\pm$6.3	98.7 $\pm$ 6.4	67.0 $\pm$ 26.6	75.3 $\pm$ 14.8	67.7 $\pm$ 21.0	89.7$\pm$12.6
$n = 10$	94.5 $\pm$ 9.8	97.0 $\pm$ 2.0	93.3 $\pm$ 9.1	100.0$\pm$1.3	62.7 $\pm$ 25.5	66.6 $\pm$ 15.6	63.3 $\pm$ 25.7	83.5$\pm$13.5
$n = 15$	82.0 $\pm$ 26.9	93.3 $\pm$ 9.1	88.4 $\pm$ 9.5	99.5$\pm$6.8	65.0 $\pm$ 26.0	64.8 $\pm$ 25.7	39.6 $\pm$ 17.2	71.3$\pm$14.3

their performances exhibit a sharp decline, even being surpassed by the base method, as n increases. Conversely, our proposed BiTrajDiff generally achieves the highest scores and consistently maintains its advantage as the n -step horizon increases. Meanwhile, as described in Section 5.4, the performance of all tested methods degrades with an increasing n -step horizon, which is attributed to the escalating variance in value estimation. Moreover, for DA methods, the accumulated errors introduced by the inverse dynamics model should also be considered. In contrast, BiTrajDiff alleviates these issues by synthesizing novel, high-quality long-horizon trajectories, thereby achieving a more stable and consistent performance. This result demonstrates that by stitching forward-future and backward-history trajectories, BiTrajDiff synthesizes higher-quality and more reliable trajectories than existing DA methods.

I. Ablation Study on Horizon Lengths

We further investigate the influence of the generation horizon length H in BiTrajDiff on *walker2d-medium-replay-v2*. Since BiTrajDiff performs bidirectional generation around the anchor state, the effective bridge span after stitching becomes $2H - 2$ transitions. The corresponding results are shown in Figure S2. The results show that a moderate horizon length provides the best trade-off between trajectory diversity and dynamics reliability. Specifically, increasing the horizon from $H = 3$ to $H = 5$ substantially improves the downstream Offline RL performance under both IQL and TD3BC, indicating that moderate bidirectional bridge generation helps recover additional trajectory-level connectivity absent from the original dataset. However, further increasing the horizon gradually degrades performance, and excessively long horizons such as $H = 20$ lead to severe instability. This trend is also consistent with the observations reported in RTDiff. Intuitively, longer generated segments are harder to model accurately, resulting in accumulated modeling errors in the synthesized trajectories, which further propagate through downstream TD estimation and value learning. Therefore, although larger bridge spans may introduce richer connectivity patterns, overly long horizons reduce the reliability of the generated data and ultimately harm offline policy optimization.


 Figure S2. Ablation study on H .

J. Time Overhead

We measure the runtime overhead during both the training and generation stages of BiTrajDiff and other diffusion-based DA methods, as shown in Figure S3. The comparison is conducted on a single 80GB A100 GPU under the *halfcheetah-medium-replay-v2* setting. The results show that BiTrajDiff maintains training and generation runtimes within a comparable range to other diffusion-based DA baselines.

K. Theoretical Intuition for Bidirectional Stitching

BiTrajDiff is motivated by the trajectory-level connectivity gap between single-direction and bidirectional trajectory modeling paradigm. Consider an offline dataset $\mathcal{D}$ consisting of trajectories $\tau \in \mathcal{D}$. For a given anchor state s , let h^- denote a local history segment arriving at s , and let h^+ denote a local future segment departing from s . We define the incoming and

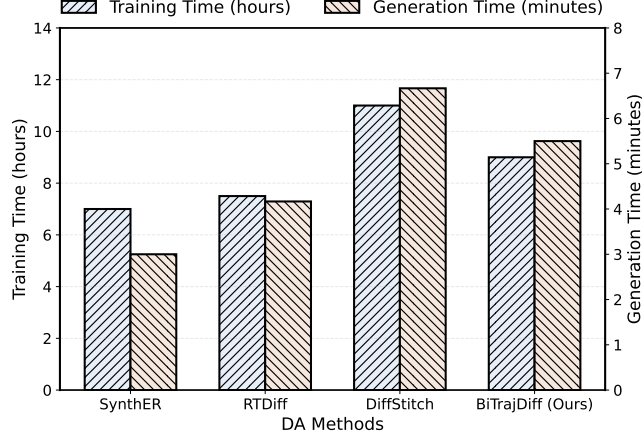


Figure S3. Runtime comparison of BiTrajDiff and other DA methods.

outgoing local pattern sets around s as

$$\mathcal{H}_{\mathcal{D}}(s) = \{h^- : \exists \tau \in \mathcal{D}, (h^-, s) \subset \tau\}, \quad (7)$$

$$\mathcal{F}_{\mathcal{D}}(s) = \{h^+ : \exists \tau \in \mathcal{D}, (s, h^+) \subset \tau\}. \quad (8)$$

Although $\mathcal{H}_{\mathcal{D}}(s)$ and $\mathcal{F}_{\mathcal{D}}(s)$ may both be well supported locally, the dataset only observes a subset of their trajectory-level couplings:

$$\mathcal{C}_{\mathcal{D}}(s) = \{(h^-, h^+) : \exists \tau \in \mathcal{D}, (h^-, s, h^+) \subset \tau\} \subseteq \mathcal{H}_{\mathcal{D}}(s) \times \mathcal{F}_{\mathcal{D}}(s). \quad (9)$$

In many offline datasets, $\mathcal{C}_{\mathcal{D}}(s)$ can be much sparser than the full product space $\mathcal{H}_{\mathcal{D}}(s) \times \mathcal{F}_{\mathcal{D}}(s)$. **Therefore, the missing support is not necessarily missing local state coverage, but rather missing support in the joint connectivity structure between incoming and outgoing trajectory patterns.**

The cross-door setting as shown in Figure 1 provides a controlled example of this phenomenon. Doorway-adjacent states s_d can have sufficient one-sided support, meaning that both $\mathcal{H}_{\mathcal{D}}(s_d)$ and $\mathcal{F}_{\mathcal{D}}(s_d)$ contain plausible local segments. However, the observed coupling set $\mathcal{C}_{\mathcal{D}}(s_d)$ may occupy only a small portion of $\mathcal{H}_{\mathcal{D}}(s_d) \times \mathcal{F}_{\mathcal{D}}(s_d)$. This makes the task suitable for trajectory-level stitching: success depends less on extrapolating to unseen local states and more on recovering unseen pairings between locally supported behavior patterns.

This perspective also explains why single-direction data augmentation can be limited in such settings. A single-direction generator is trained on continuations sampled from trajectories in $\mathcal{D}$, so its supervision is concentrated on the observed coupling structure $\mathcal{C}_{\mathcal{D}}(s)$. If a cross-pattern pairing (h^-, h^+) is absent from $\mathcal{D}$, it receives little direct training signal, and the learned generator tends to preserve the empirical connectivity structure rather than create new pairings. In contrast, BiTrajDiff factorizes bridge generation around the same anchor into two conditional generation problems: a backward-history model for h^- and a forward-future model for h^+ . This factorization allows candidate pairings to be drawn from the richer product space induced by $\mathcal{H}_{\mathcal{D}}(s) \times \mathcal{F}_{\mathcal{D}}(s)$ instead of being restricted to $\mathcal{C}_{\mathcal{D}}(s)$. The inverse dynamics completion and trajectory filters then retain feasible and high-quality candidates. This provides the main theoretical intuition for why bidirectional generation is effective at recovering missing trajectory-level connectivity than other single-direction methods.